

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

A battery management system (BMS) protects a lithium-ion pack and estimates its state of charge. It already holds the three measurements from which ageing is visible: terminal voltage, current and temperature [1]. A software layer above it can, in principle, turn those measurements into answers that an operator or driver can act on: how much of its original capacity the battery retains (state of health, SOH), how fast that is changing, how long it has before it should be replaced (remaining useful life, RUL), and what usage is doing to it.

Capacity and resistance are the two quantities through which ageing usually appears. Capacity fades as lithium inventory and active material are lost, and internal resistance grows, which reduces the power the cell can deliver [3, 4]. Reviews of SOH estimation distinguish direct measurement, model-based estimation and data-driven approaches [2, 1]. A vehicle, however, rarely provides the conditions a laboratory does: discharges are partial, rates vary, and no reference test is available. Any practical health estimator must therefore work from what the vehicle actually records.

1.2 MOTIVATION

The dominant way to build such a software layer is to fit a model. Features are extracted from cycling data and a regressor is trained to predict SOH, with feature sets ranging from usage aggregates to discharge-curve descriptors such as the capacity–voltage difference between cycles [6]. Skill is commonly reported under leave-one-cell-out validation, in which a held-out cell’s siblings from the same test protocol remain in the training set, so the test asks whether the model recognises a protocol it has already seen.

A deployed BMS faces a harder question: the next battery comes from conditions the training set never contained. This project began as a battery-degradation study built on fitted models, and measured that question directly. On a benchmark of 31 NASA cells in nine protocol cohorts, eleven of twelve methods lost skill, and the twelfth, a mean-only baseline, could not when the held-out unit changed from a cell to a whole protocol cohort, and no method could be distinguished from another (Chapter 5). That negative result is the motivation for what follows: if models fitted across cells do not transfer, a health layer for a vehicle has to be built from estimators that fit nothing across cells, and from a pipeline that is honest about when it does not know.

The work is also motivated by engineering practice. A monitoring system that is asked for a number it cannot compute and returns a plausible one is dangerous. A defect of exactly this kind occurred during development: a missing temperature channel was scored as “temperature is fine”, because a comparison against NaN evaluates to false, so every heat check silently passed for a pack nobody had measured. The design of BEACON takes the opposite stance: the system refuses to produce a number it cannot justify, and says why.

1.3 PROBLEM STATEMENT

The problem addressed in this project is how to build a software layer above a BMS that can measure battery health from the BMS’s own signals, namely voltage, current and time, without depending on a model trained on other cells, and that knows when its evidence is insufficient. Three sub-problems follow.

- **Measurement.** How can SOH, internal resistance and RUL be measured from voltage, current and time alone, including from partial discharges, and how accurately?
- **Honesty.** How can the system decide, without ground truth, when an estimate should be reported with a stated error band and when it should be refused?
- **Integration.** How can data from a CAN bus, a serial bench rig and research datasets all be scored through one shared pipeline and served to a dashboard, so that a difference in output is a difference in the telemetry and not in the code?

1.4 RESEARCH GAP

Data-driven SOH estimators are widespread, but how they are validated changes what they appear to achieve. Group-wise splitting by cell changes reported skill substantially [7], and a large gap between random and cell-wise splits has been reported on NASA data [8]. These studies stop at the cell. What remains insufficiently characterised is the next step: whether skill survives a change of *test protocol*, and what a BMS software layer can still measure reliably when it does not.

Curve-based methods, including differential voltage analysis [9] and incremental capacity analysis, including on-board use [11], read degradation from the shape of the voltage–charge curve. Online recursive estimation of cell parameters, including resistance, is established [5]. The gap this project addresses is the combination: estimators validated from raw telemetry rather than from laboratory summaries, with an online resistance correction taken from the load step, gates that refuse rather than extrapolate, and a measured criterion for when the evidence suffices, all evaluated on more than one dataset without re-tuning.

1.5 OBJECTIVES

The objectives of the project are:

1. To build an end-to-end telemetry system that ingests data from a CAN bus, a serial bench rig and research datasets, converges them on one schema, and scores them through one shared pipeline.
2. To estimate SOH from voltage, current and time alone, with an ohmic correction identified from the load step, and to validate it against laboratory capacity.
3. To report resistance as a second validated health quantity, and to estimate RUL from each cell's own fade trend, reported as a user meets it.
4. To measure rather than assert when the evidence suffices, so that every output is validated, banded by its measured error, or refused.
5. To validate externally, without re-tuning, on a second and third dataset, and to retain negative results.
6. To build and bring up a physical data-acquisition rig, and to serve all results through a REST API and a web dashboard.

1.6 BATTERY HEALTH INDICATORS

Three quantities are used throughout this report, and they are defined here once so that every later chapter uses them in the same way.

1.6.1 State of Health

State of health is the fraction of the original capacity that the cell can still deliver. With Q_{now} the capacity measured on a full constant-current discharge and Q_{ref} the reference capacity of the same cell, it is

$$\text{SOH} = \frac{Q_{\text{now}}}{Q_{\text{ref}}} \times 100 \%. \quad (1.1)$$

The choice of Q_{ref} matters. It may be the rated capacity from the data sheet or the capacity measured on the cell's own first discharges. The two give different numbers for the same cell, and this report states which is used wherever a figure is quoted. A battery is conventionally regarded as having reached end of life when SOH falls to 80%. The RUL estimator of this project uses a default threshold of 90%, because the CALCE trajectories rarely reach 80% within the recorded life, and reports the 80% case separately where the data allow it.

1.6.2 Internal Resistance

Internal resistance is the ratio of the voltage step to the current step when a load is applied. For a step from current I_1 to I_2 that moves the terminal voltage from V_1 to V_2 ,

$$R_{\text{int}} = \frac{|V_2 - V_1|}{|I_2 - I_1|}. \quad (1.2)$$

Resistance rises as the cell ages, and it determines the voltage sag under load and therefore the power the cell can deliver.

1.6.3 Remaining Useful Life

Remaining useful life is the number of further cycles before SOH reaches the end-of-life threshold. It is a prediction, so it carries an error that grows with the distance from the threshold, and a figure for it is meaningful only if the error is stated alongside it.

1.7 PROPOSED APPROACH

BEACON, the system built in this project, is organised in three tiers. The data tier receives telemetry from three sources, a controller area network (CAN) bus, a serial link from a bench rig and public research datasets, and converts all three into one unified frame. The analytics tier is a Python service that scores every frame through a single function and returns state of health, resistance, remaining useful life and a behaviour risk score, each with its evidence and, where the evidence is insufficient, a refusal with the reason. The presentation tier is an Express gateway and a React dashboard that display the results and label every value with its source.

Three design commitments run through the work. The first is that nothing is fitted across cells: the estimators measure each cell against its own early behaviour. The second is that a result is either validated against laboratory truth, reported with a measured error band, or refused. The third is that the same scoring function serves a vehicle, a rig and a dataset, so that any difference between their outputs comes from the telemetry and not from the code.

1.8 SIGNIFICANCE OF THE WORK

For a fleet operator or an owner, a health figure is useful only if it can be trusted, and a figure that cannot be trusted is worse than none. By measuring its own error on public data and by declining to answer when its evidence is poor, the system gives an operator a reason to act on an estimate when one is shown and a reason to collect more data when one is withheld. For the research community, the retained negative results, in particular the loss of skill of fitted models when the test protocol changes, are a reusable finding about how battery health models should be validated. For students and practitioners, the repository is a worked example of how a software layer over a BMS can be tested,

documented and reproduced.

1.9 SCOPE AND ORGANISATION OF THE REPORT

Development and primary validation use single lithium cobalt oxide (LCO) cells at constant current (CALCE). External validation uses NMC/LCO blend pouch cells at 40 °C (Oxford), which also supply one real drive-cycle discharge. A third dataset (NASA) is used for the benchmark and for a cross-dataset test. The project makes no claim about packs, lithium iron phosphate, or temperature-dependent capacity correction. The bench rig validates data acquisition, not health estimation: no discharge has yet been recorded on it.

Table 1.1 gives a map of the chapters. The remainder of the report is organised as follows. Chapter 2 reviews related work. Chapter 3 presents the system design and the estimation methods. Chapter 4 describes the implementation, the hardware rig and the deployment. Chapter 5 reports the results and discusses them. Chapter 6 concludes and lists future work.

Table 1.1: Organisation of the Report

Chapter	Title	Content
1	Introduction	Background, problem, objectives and scope
2	Literature Review	Ageing, estimation methods, interfaces and validation practice
3	Methodology / System Design	Requirements, schema, estimators, gates and validation protocol
4	Implementation and Deployment	Acquisition, rig, services, security and testing
5	Results and Discussion	Benchmark, estimator accuracy, hardware results and threats to validity
6	Conclusion and Future Work	Findings, limitations, lessons and next steps

CHAPTER 2

LITERATURE REVIEW

2.1 STATE ESTIMATION IN THE BATTERY MANAGEMENT SYSTEM

A BMS protects the pack and estimates its state of charge from the terminal measurements it already holds. Reviews of monitoring methods for electric and hybrid vehicles [1] and of SOH estimation for real applications [2] distinguish direct measurement (capacity or resistance tests), model-based estimation and data-driven approaches. Ageing mechanisms and their effect on capacity and resistance are reviewed by Vetter *et al.* [3] and, for automotive applications, by Barré *et al.* [4]. Recursive estimation of cell parameters within the BMS, including internal resistance, is established [5], and monitoring of resistance is part of the methods reviewed by Waag *et al.* [1].

Textbook treatments of the BMS, including state-of-charge indication and cell balancing, are given by Pop *et al.* [21] and Plett [22]. State of charge and state of health are related but different quantities: the first says how much charge remains in the present state of the cell, the second says how much the cell can hold at all, and a layer that reports health must not take its reference from a state-of-charge estimate that itself depends on a capacity assumption.

In this project, resistance serves two roles: it corrects the ohmic sag that would otherwise make a fixed voltage window slide along the discharge curve, and it is reported as a second health output, validated against the laboratory instrument (Section 5.4).

2.2 DATA-DRIVEN SOH ESTIMATION AND ITS VALIDATION

Data-driven estimators trained on cycling features are widespread. Severson *et al.* [6] showed that features of the capacity–voltage curve, such as the variance of the difference between cycles, predict cycle life from early data. Such models are commonly assessed under random or leave-one-cell-out splits.

How they are validated changes what they appear to achieve. Group-wise splitting by cell reduces reported skill substantially [7], and a large gap between random and cell-wise splits has been reported on NASA data [8]. This project extends the hold-out from the cell to the test protocol, using leave-one-cohort-out validation with a bootstrap interval over folds, and finds that the interval on any single method is wider than the spread between methods (Section 5.1).

2.3 DISCHARGE-CURVE ANALYSIS

Differential voltage analysis [9] and incremental capacity analysis, including on-board use [11], read degradation from the shape of the voltage–charge curve, and their features can be mapped to degradation modes [10, 12]. Estimators using partial charging curves have also been reported [13].

The window estimator used here belongs to this curve-based family. It differs in four ways: it uses no cross-cell fitting, it applies an online resistance correction from the load step, it uses gates that refuse rather than extrapolate, and it is validated from raw telemetry rather than from laboratory summaries.

2.4 REMAINING USEFUL LIFE PROGNOSTICS

Remaining-useful-life methods and their uncertainty are reviewed by Hu *et al.* [14]. Accuracy is often summarised as a single error figure over all predictions. This project reports RUL error indexed by the prediction a user actually sees, for example “when the system predicts fewer than 25 cycles, how often did the cell last at least that long”, and records which routes to RUL failed (Sections 5.5 and 5.6).

2.5 PUBLIC DATASETS

Three public datasets are used, chosen because they differ in chemistry, format, temperature and protocol.

- **CALCE CS2 and CX2** [16]: prismatic LCO cells cycled at room temperature under several protocols, including multi-rate cycling and real partial cycling. Primary development and validation.
- **NASA PCoE** [15]: 18650 cells under nine documented protocols with ambient temperature from about 4 to 43 °C. Used for the benchmark and for a cross-dataset test.
- **Oxford Battery Degradation Dataset 1** [17, 18]: eight Kokam pouch cells aged at 40 °C with an Artemis urban drive-cycle discharge. Used for external validation.

2.6 TELEMETRY ACQUISITION IN VEHICLES AND LABORATORY RIGS

In a vehicle, BMS data is carried on a CAN bus, and a DBC file describes how signals are packed into frames. A research dataset arrives as files, and a bench rig typically streams text over a serial port. Each source has its own failure modes: a CAN decode can silently miss a signal, a serial link can drop or corrupt bytes, and a research file can store a quantity with a different sign or scale. The review above does not address how these three sources are made to agree, and this project treats it as a design question (Chapter 3): one schema, one scoring function, and explicit refusals when a capture’s shape is wrong.

2.7 BATTERY AGEING MECHANISMS

The capacity and resistance that the estimators in this project measure are the visible result of physical processes inside the cell. Growth of the solid electrolyte interphase on the negative electrode consumes lithium inventory and thickens the resistive film, so capacity falls and resistance rises. Lithium plating, which is more likely at high charge rates and low temperature, removes lithium irreversibly and can create a safety risk. Loss of active material through particle cracking or dissolution reduces the number of sites available to hold lithium [3, 4]. These mechanisms depend on temperature, state of charge, current rate and depth of discharge, which is why a monitoring layer pays attention to how a battery is used and not only to its age. They also explain why a single protocol, such as constant-current cycling at room temperature, cannot represent the conditions a vehicle imposes, a point on which the validation in Chapter 5 depends.

2.8 TAXONOMY OF HEALTH ESTIMATION METHODS

Reviews of the field group state-of-health estimation into three families [2, 1]. Table 2.1 summarises them and states what each requires, because the requirement, more than the accuracy, decides whether a method can run inside a vehicle.

Table 2.1: Families of State-of-Health Estimation

Family	Principle	Requirement in a vehicle
Direct measurement	Measure capacity or resistance by a dedicated test	A controlled full discharge or load pulse, rarely available
Model-based	Fit an equivalent-circuit or electrochemical model and track its parameters	Accurate model, parameter identification and a good initial state
Data-driven	Learn a mapping from features to health from past cells	Training cells representative of the deployment conditions

The window estimator of this project is a direct measurement made opportunistically: whenever the telemetry contains a discharge that is long enough, it counts the charge delivered between two voltage thresholds and reads capacity from that, so it needs no dedicated test and no model of other cells.

2.9 INTERNAL RESISTANCE ESTIMATION IN THE LITERATURE

Internal resistance can be measured from the voltage response to a current step, from electrochemical impedance spectroscopy or from recursive parameter identification inside an equivalent-circuit model [5, 1]. The step method is the simplest and is the one adopted here, because a vehicle produces current steps routinely, at every transition between acceleration, cruising and braking. The step response contains an instantaneous ohmic part and slower diffusion and charge-transfer parts, so a resistance read from the

first loaded sample is an apparent value that includes some polarisation. This project therefore treats the estimate as a trend indicator whose direction is validated against the laboratory instrument, and uses it as a correction term for the voltage window, and not as an absolute impedance.

2.10 VEHICLE DATA INTERFACES

2.10.1 Controller Area Network

The controller area network is the serial bus on which the control units of a vehicle exchange short frames [19]. A frame carries an identifier and up to eight data bytes, and the identifier determines which message the data belong to. The bytes are packed with several signals, each defined by a start bit, a length, a byte order, a scale factor and an offset. A DBC file records these definitions. Manufacturers regard their DBC files as proprietary, which is the main obstacle to a general decoder, and public decoding exists for a limited number of vehicles through community projects such as the Open Vehicles Monitoring System.

2.10.2 Serial Laboratory Rigs

A laboratory or student rig usually streams text lines over a universal asynchronous serial link. Such a link has no error detection of its own, so bytes can be lost or corrupted, and a boot banner or debug message can appear between data records. A reliable acquisition layer therefore needs a delimiter, a checksum and a rule for what to do with a record that fails them. These concerns do not arise with a dataset file, which is why the three sources need an explicit common schema.

2.10.3 Open Datasets

Public datasets store the same physical quantities under different names, units, sign conventions and file formats. Reported capacity may be in ampere-hours or in coulombs, current may be positive on charge or on discharge, and temperature may be absent. Without a documented mapping into a single schema, a result computed on one dataset cannot be compared with a result on another.

2.11 DIGITAL TWINS AND FLEET MONITORING

A digital twin is a virtual representation of a physical asset that is updated from the asset's data and can be queried in its place [20]. For batteries, the term is used for anything from a detailed electrochemical simulation to a simple state record. In BEACON the twin is deliberately modest: a state machine layered on the outputs of the scoring pipeline that records the current health state of each battery and every transition between states across runs. It adds structure and a queryable history and does not add a predictive model, and the report says so wherever the twin is mentioned. In a fleet, such a record lets an

operator see which batteries have changed state since the last evaluation, which is what a maintenance decision depends on.

2.12 EXPLAINABILITY AND OUTPUT TRANSPARENCY

Explanations of a model’s output are expected in safety-relevant systems, and feature attribution is the usual tool. For rule-based scores, the contribution of each rule to the total can be computed exactly and does not need an approximation. The behaviour layer of BEACON reports such attributions for its heuristic scores. A related concern is the honesty of the output itself. A dashboard that displays a placeholder, a default or an extrapolated value looks identical to one that displays a measurement. This project therefore treats the provenance of every displayed number as part of the result, and the dashboard renders only computed values, labelling those that come from simulated data.

2.13 VALIDATION PRACTICE AND DATA LEAKAGE

The way data are split between training and testing determines what a reported accuracy means. A random split over cycles allows neighbouring cycles of the same cell to appear on both sides, which inflates accuracy. A leave-one-cell-out split removes that leak but still places the held-out cell’s siblings from the same protocol in the training set. A leave-one-cohort-out split removes the whole protocol, and so tests transfer to conditions that were not seen. Table 2.2 summarises the three, and the benchmark in Chapter 5 uses the last two.

Table 2.2: Validation Splits and Their Purpose

Split	What is held out	What the score tests
Random over cycles	Individual cycles	Interpolation within seen cells
Leave-one-cell-out	One whole cell	Recognition of a seen protocol
Leave-one-cohort-out	All cells of one protocol	Transfer to an unseen protocol

2.14 COMPARISON OF APPROACH FAMILIES

Table 2.3 summarises the three families of approach in this review by the properties that matter for a deployed software layer. It is a statement about the families as categories and not about any individual cited study.

2.15 IDENTIFIED RESEARCH GAPS

Table 2.4 lists the gaps that the review leaves open and the parts of this report that address them.

Table 2.3: Comparison of Approach Families

Property	Fitted regression	Curve-feature methods	This work (BEACON)
Needs a model trained on other cells	Yes	Varies	No
Reference needed for the same cell	No	Varies	First discharges only
Typical hold-out unit in validation	Cell	Varies	Whole protocol cohort, with interval
Refuses when evidence is missing	Not by design	Not by design	Yes, with stated cause
Works from partial discharges	Varies	Varies	Yes, with a knee gate

Table 2.4: Research Gaps and Corresponding Report Sections

Gap	Addressed in
Skill of fitted models under a change of protocol	Benchmark with leave-one-cohort-out validation (Section 5.1)
Health estimation without cross-cell fitting	Window estimator with ohmic correction (Chapter 3)
Refusal when evidence is missing or inconsistent	Refusal gates and evidence sufficiency (Chapters 3 and 4)
Agreement of CAN, serial and file sources	Unified frame and single scoring function (Chapter 4)
Provenance of displayed values	Dashboard that renders computed values only (Chapter 4)

2.16 SUMMARY

The literature establishes the measurements, the curve-based family of methods, online resistance estimation and the importance of validation splits. What it leaves open, and what this project addresses, is a software layer that is validated from raw telemetry across protocols and datasets, that fits nothing across cells, and that returns a refusal instead of a number when its evidence is insufficient.

CHAPTER 3

METHODOLOGY / SYSTEM DESIGN

This chapter explains *what* was designed, *why* it was designed in this way, and *how* the experiments were controlled. The practical realisation of the design (code organisation, wire protocol, hardware rig, services and deployment) is described in Chapter 4.

3.1 PROBLEM AND DESIGN OBJECTIVE

The objective is a software layer above a BMS that converts voltage, current and time into health outputs an operator can act on, with two design constraints that shape everything below.

1. **Fit nothing across cells.** Chapter 5 shows that models fitted across cells do not retain skill under a change of test protocol. Both working estimators (SOH and RUL) are therefore expressed relative to the same cell's own early behaviour, so there is no training cohort to transfer from.
2. **Refuse what cannot be justified.** Every output is validated, banded by its measured error, or refused with a stated cause. A refusal is a correct outcome that the system knows, so it is returned as a value, rendered, and reflected in the process exit code.

3.1.1 Research Variables

- **Independent variables:** the ohmic correction used (none, cyclor resistance, resistance from the load step), the voltage window (fixed or learned), the reference size, the RUL threshold, and the dataset (CALCE, Oxford, NASA).
- **Dependent variables:** per-cell mean absolute SOH error and its median over cells, coverage (cells measured out of cells scored), rank correlation of resistance, RUL error indexed by true and by predicted distance to end of life, and error stratified by the consistency of readings.
- **Controlled variables:** the reference definition (a quantity a BMS can hold), the primary window and tolerances, fixed before each study ran, and the scoring rule. Choices made after seeing a result are labelled exploratory.

3.2 SYSTEM REQUIREMENTS

The requirements below were fixed before the implementation and are the basis on which the results of Chapter 5 are judged. Table 3.1 lists them.

Table 3.1: Functional and Non-Functional Requirements

ID	Requirement	Type
R1	Accept telemetry from a CAN bus, a serial rig and dataset files	Functional
R2	Convert every source to one schema with documented units and sign convention	Functional
R3	Estimate SOH, resistance and RUL from voltage, current and time	Functional
R4	Refuse, with a stated reason, when the evidence is insufficient	Functional
R5	Serve all results over an HTTP API and a web dashboard	Functional
R6	Fit no model across cells for the shipped estimators	Design constraint
R7	Show the provenance of every displayed value	Non-functional
R8	Run end to end from a clean checkout with one command each for tests and results	Non-functional
R9	Need no external service or physical hardware in continuous integration	Non-functional

3.3 ASSUMPTIONS AND CONSTRAINTS

The design rests on a small number of explicit assumptions. Telemetry is sampled at a known and reasonably uniform rate, so that charge can be obtained by integrating current over time. The current sign convention of each source is known and is corrected in the loader. A cell's own early discharges are representative of its undegraded state, which is why the reference is taken from them when the rated capacity is not declared. The service is a local bench and analysis tool: it binds to the loopback address by default, has no authentication and keeps its state in memory. These constraints are recorded because several of the results and several of the design choices follow from them.

3.4 OVERALL SYSTEM FRAMEWORK

BEACON ingests telemetry from three transports, converges them on one schema, and scores them through one shared function. The result, including every refusal, is served over an HTTP API to a React client (Fig. 3.1).

3.4.1 Unified Scoring Function

The function `score_telemetry_frame` contains everything downstream of acquisition: segmentation, coulomb counting, features, risk, health, RUL, the Guardian explanation and the digital twin. It was extracted from the CAN pipeline when serial ingestion was added, instead of being copied. A second segmentation or feature extractor would mean that a disagreement between a CAN run and a serial run over the same battery could not be traced to either the data or the code. With the tail shared, a difference

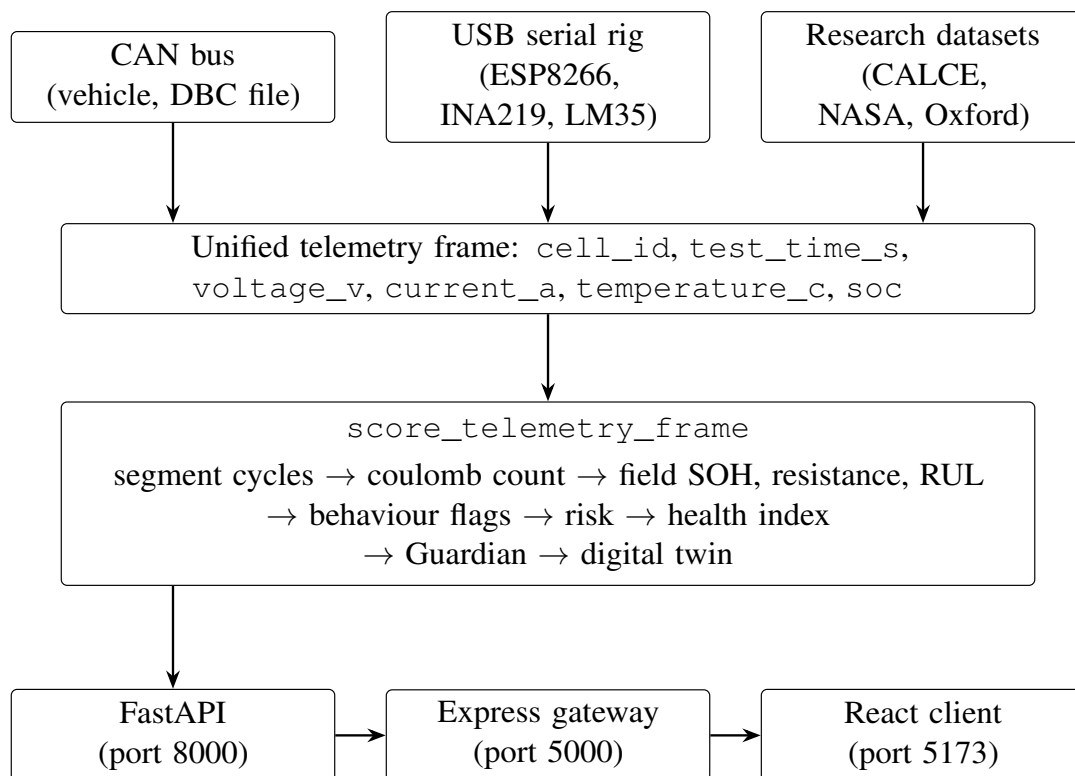


Figure 3.1: Overall system framework: three transports, one unified frame, one scoring function, and the service tier.

in output is a difference in the telemetry.

3.5 DATASETS

Four sources are used (Table 3.2). They were chosen because they differ in chemistry, format, temperature and protocol.

Table 3.2: Data Sources and Their Role

Source	Cells and conditions	Role	Ground truth used
CALCE CS2/CX2	22 prismatic LCO cells, room temperature, 7 protocols (constant-current, multi-rate, real partial cycling)	Development and primary validation	Cycler-measured discharge capacity
NASA PCoE	18650 cells, 9 protocols, ambient 4–44 °C; 31 cells in the benchmark, 27 in the cross-dataset study	Benchmark and third-dataset test	NASA’s computed capacity
Oxford Dataset 1	8 Kokam NMC/LCO pouch cells, 740 mAh, 40 °C, 519 characterisations	External validation, unchanged	1C capacity at each characterisation
Bench rig	1 HONGLI 18650 cell, at rest in the two real captures (83 and 85 records)	Data acquisition only	Frame integrity and timing

CALCE records span many files whose test time restarts per file. A monotonic time base is constructed by offsetting each restart, not by sorting timestamps, which would interleave overlapping segments. The CALCE set includes real partial cycling (CS2 Types 5 and 6): thousands of discharges of about 25% depth in a fixed band, with a full capacity check roughly every 100 cycles. Type 6 cycles at the top of charge (about 4.07 to 3.78 V) and Type 5 near empty (about 3.69 to 2.70 V). The Oxford set additionally holds one real drive-cycle discharge of Cell 1 sampled at 1 Hz, used only for external validation.

3.6 UNIFIED TELEMETRY SCHEMA

Public datasets, vehicle frames and a serial rig describe the same measurements with different names, units and missing-value conventions, so every source is converted into one schema before any feature is computed. Table 3.3 lists the core columns.

Table 3.3: Core Columns of the Unified Schema

Column	Unit	Required	Description
dataset	text	Yes	Source dataset name
cell_id	text	Yes	Cell, module or vehicle identifier
cycle or timestamp	–	One of two	Ordering of the samples
voltage_v	V	Yes	Terminal voltage
current_a	A	Yes	Current with a documented sign
temperature_c	°C	Yes	Battery temperature
capacity_ah	Ah	Recommended	Measured capacity
soc	%	Recommended	State of charge, 0–100
resistance_ohm	Ω	Optional	Internal resistance if available

3.6.1 Derived Fields

Three fields are derived from the core columns and regenerated after every preprocessing run. Power is the product of voltage and current. The operating mode is inferred from the current when the source does not give one: a current above +0.02 A is charge, below –0.02 A is discharge, and a current inside that band is rest. The state-of-charge band assigns each sample to one of four ranges, below 20% (deep-discharge region), 20 to 80% (preferred region), 80 to 90% (elevated) and 90% and above (high-state-of-charge region, where calendar ageing is greatest). These bands feed the behaviour layer of Section 3.15.

3.6.2 Validation of the Schema

After conversion the validator checks that the required columns exist and are not entirely empty, that numeric columns can be converted to numbers, that state of charge and state of health lie between 0 and 100 when present, that voltage, capacity and resistance are not negative, and that either a cycle or a timestamp exists. The validator reports each failure with its reason. For the CAN and serial paths, the stop for a missing channel is a separate signal-coverage check, which compares the channels a source provides with the channels the estimators need and refuses the run by name, as when a CAN message carries current and state of charge but no voltage or temperature. The serial frames do not carry a dataset name, so the required-column rule applies to the dataset loaders and not to the live telemetry paths.

3.7 DATA FLOW AND PROCESSING SEQUENCE

Every source follows the same eight steps in the same order. The CAN and serial paths run them through the single function `score_telemetry_frame`, and the dataset studies run the same estimators through the study scripts and the batch pipeline.

1. **Acquire.** Read frames from the bus, lines from the serial port or rows from a file.
2. **Convert.** Decode signals or fields and convert to the units and sign convention of the schema.
3. **Validate.** Apply the schema checks and the capture-level gates, and refuse the capture if they fail.
4. **Segment.** Split the stream into discharges by current direction and by gaps in time.
5. **Measure.** Integrate current to obtain charge between voltage thresholds, apply the ohmic correction and estimate SOH and resistance.
6. **Predict.** Extrapolate the cell's own fade to the end-of-life threshold and attach an error band.
7. **Score behaviour.** Compute the stress features, the risk score, the health index and the Guardian explanation.
8. **Publish.** Update the twin and the fleet store, and serve the result with its evidence and its refusals.

3.8 DISCHARGE SEGMENTATION

The estimator receives voltage $V(t)$, current $I(t)$ and time. The cyclers's capacity, resistance and cycle summaries are removed before scoring. Current is segmented into charge, discharge and rest phases using a dead band (0.02 A in the CALCE studies, where rest is logged as exactly zero, and 0.5 A as the default of the live telemetry path) and a minimum run length. The charge delivered on discharge n is the trapezoidal integral of $|I| dt$. Every discharge is retained: a partial discharge that crosses the window is a full measurement of the window.

3.9 VOLTAGE-WINDOW SOH ESTIMATION

For a window $[V_{\text{low}}, V_{\text{high}}]$, the window charge $Q_w(n)$ is the charge delivered while terminal voltage falls from V_{high} to V_{low} on discharge n . It is read by interpolation and requires that the discharge span at least 95% of the window. With $\mathcal{R}$ the cell's first five measurable discharges,

$$\text{SOH}(n) = \frac{Q_w(n)}{\text{median}_{m \in \mathcal{R}} Q_w(m)}, \quad (3.1)$$

and the reported value is the median of the latest five accepted readings. The primary window, 3.90–3.60 V, was fixed before any validation run. The windows 4.05–3.55 and 3.85–3.55 V are reported as sensitivity analyses and not selected between.

Equation (3.1) assumes the discharge curve scales approximately uniformly with capacity. Loss of lithium inventory shifts the curve, which the ratio partly absorbs. Loss of active material and resistance growth change its shape and position, which it does not. This is the method's principal approximation, and Section 5.7 measures how it drifts with age. Fig. 3.2 summarises the estimator.

3.10 INTERNAL RESISTANCE ESTIMATION AND OHMIC CORRECTION

Under load, terminal voltage lies below open-circuit voltage by approximately IR . As R grows with age, a fixed terminal-voltage window slides along the curve, and the slide reads as additional fade. At each discharge onset, between the last rest sample and the first loaded sample,

$$R_{\text{step}} = \frac{V_{\text{rest}} - V_{\text{load}}}{|I_{\text{load}} - I_{\text{rest}}|}, \quad (3.2)$$

accepted when the two samples are within 60 s and the current change is at least 0.05 A. R_{step} is an *apparent* resistance: the ohmic drop plus polarisation accrued by the first loaded sample. On cell CS2_35 its starting value is about 0.141 Ω against the cycler's about 0.089 Ω (reports/metrics/calce_resistance). Each discharge uses the trailing median $\tilde{R}(n)$ of the last 11 step estimates, so no correction depends on later data, and its window

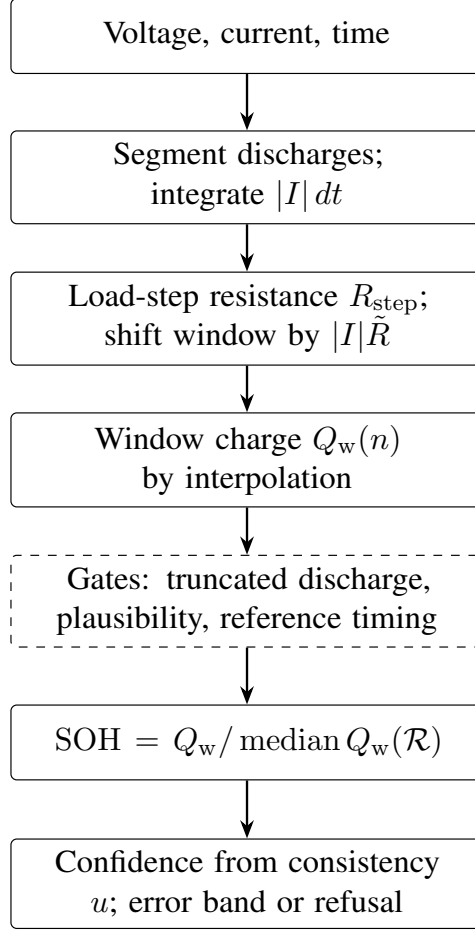


Figure 3.2: Field SOH estimator: from raw telemetry to a banded or refused output.

is shifted to

$$[V_{\text{low}} - |I_n| \tilde{R}(n), V_{\text{high}} - |I_n| \tilde{R}(n)]. \quad (3.3)$$

Discharges preceding the first resistance estimate are left unmeasured rather than scored uncompensated.

3.11 DATA-QUALITY GATES AND REFUSAL CONDITIONS

Each gate needs no ground truth and can therefore run in deployment.

1. **Truncated discharge.** A discharge delivering less than 50% of the reference discharges' total charge is not scored. A cut-short cycle crosses the window with little charge and would read as severe fade.
2. **Physical plausibility.** A ratio below 0.50 is withheld, and a cell with more than 20% of readings below it is refused entirely. A ratio above 1.10 is withheld as not spanning the same part of the curve.
3. **Reference timing.** The reference must be complete within 20 equivalent full cycles of charge throughput. Throughput rather than a discharge count is used because storage-protocol records contain thousands of short pulses.

3.12 LEARNED VOLTAGE WINDOWS FOR PARTIAL DISCHARGES

A car rarely discharges across a fixed window. When the fixed window cannot be read and the rated capacity C_{rated} is known, a window is learned from the cell's first ten discharges only. The band they all cover is trimmed by 15% at each end, and the flattest sub-window of up to 0.30 V within it is selected by its steepness

$$s = \frac{V_{\text{high}} - V_{\text{low}}}{Q_{\text{w}}/C_{\text{rated}}} \quad [\text{V per unit state of charge}]. \quad (3.4)$$

The window is refused if even the flattest choice has $s > 2.0$, because it then lies on the end-of-discharge knee. There, voltage reflects kinetics (resistance and solid-state diffusion) rather than stored charge.

3.13 REMAINING USEFUL LIFE ESTIMATION FROM CELL-SPECIFIC FADE

At cycle k , the cell's SOH history up to k is smoothed with an 11-point rolling median, a straight line is fitted to the full history, and its crossing of 0.90 gives the end-of-life estimate. Estimates are refused in three cases: with fewer than 30 points, for flat or rising trends, and when the estimate reaches more than twice the history length ahead. A cell already past the threshold is reported as zero. Linear, square-root and quadratic forms, over full and trailing windows, were compared, and the full-history line was most accurate near end of life. No other cell's data enters, so there is no training cohort.

3.14 EVIDENCE SUFFICIENCY CRITERION

Each output has its own requirements: capacity health needs voltage, current and five reference readings plus one; resistance health needs a clean load step; RUL needs 30 complete discharges; and heat exposure needs a temperature channel. Within capacity health, confidence follows consistency. Because a healthy estimate keeps changing as the cell ages, stability of the estimate is the wrong criterion. The question is whether readings agree once that trend is removed. With $\sigma_{\tilde{x}}(x_{1..n}) = 1.2533 \text{ MAD}(x)/\sqrt{n}$ the standard error of a median,

$$u_{\text{ref}} = \frac{\sigma_{\tilde{x}}(Q_{\text{w}}(\mathcal{R}))}{\text{median } Q_{\text{w}}(\mathcal{R})}, \quad (3.5)$$

$$u_{\text{rec}} = \frac{1.2533 \text{ MAD}(\varepsilon)}{\sqrt{5}}, \quad (3.6)$$

$$u = \sqrt{u_{\text{ref}}^2 + u_{\text{rec}}^2}, \quad (3.7)$$

where ε are the residuals of the last ten readings about their own straight line. The tolerance $u \leq 0.01$ (one SOH point) was fixed before the study of Section 5.7, chosen

to lie below the validated 1.7% median error. Confidence is then assigned as *high* (consistent, fixed window and a beginning-of-life reference), *medium* (consistent, but a learned window or a start-of-log reference), or *low* (inconsistent, reported with the measured 22-point band).

3.15 HEURISTIC BEHAVIOUR SCORING LAYER

Alongside the measured estimators, BEACON carries a rule-based behaviour layer inherited from the project’s first phase. It converts usage features into a risk score, an aging budget, a health index, a state label, a Guardian explanation and a digital-twin state. The features are average stress, average temperature, deep-discharge duration, fast-charge duration, aggressive-discharge count and average SOC.

Table 3.4: Risk Categories of the Rule-Based Layer

Risk category	Risk score
LOW	0–39
MEDIUM	40–59
HIGH	60–79
CRITICAL	80–100

Table 3.5: Health Index, Battery State and Digital-Twin State

Health index	Battery state	Twin state
0–29	HEALTHY	NORMAL
30–59	WARNING	MODERATE_RISK
60–79	DEGRADED	HIGH_RISK
80–100	CRITICAL	FAILURE_IMMINENT

The aging budget starts at 100 and is reduced by penalties for stress, temperature, deep discharge, fast charging, aggressive discharge and SOC abuse, and the health index is $HI = 100 - \text{budget}$, so a higher index means a worse battery (Tables 3.4 and 3.5). The digital twin is a one-to-one relabelling of the battery states, with a queryable API. It adds structure but no new predictive model.

This layer is *not* validated against measured degradation. The Spearman correlation between the risk score and measured fade on a separate set of 33 NASA cells from the earlier health-index ranking was $\rho = -0.27$ ($p = 0.12$), so it is labelled a heuristic wherever it appears. Where measured SOH exists and the log starts at beginning of life, the measured SOH sets the battery state and records that it did, and the heuristic index is used only otherwise.

3.16 DIGITAL-TWIN STATE MODEL

The digital twin records, for each battery, a state derived from the health index of the latest run and the history of changes in that state. The state is a relabelling of the

health index (Table 3.5), not a separately tuned threshold set, so any change to the health-index formula flows through to the twin without a second edit. A transition is recorded whenever the state of a battery differs from its state in the previous run, and a battery's first observation counts as a transition from no state. Re-running an unchanged, identically seeded fleet therefore produces no new transitions, which is checked by a test. The failure likelihood the twin reports is a monotonic transform of the health index onto the interval from 0 to 1. It is named a likelihood and not a probability because it is not a calibrated statistical quantity.

3.17 DESIGN DECISION RECORDS

Fourteen architecture decision records document the choices made during the project, with the context, the decision and its status; the table gives a short title for each. Table 3.6 lists them. Several record a claim that was withdrawn after a later measurement, and that history is kept in the repository and not rewritten.

Table 3.6: Architecture Decision Records

No.	Decision
1	Cross-dataset validation is not possible; leave-one-cohort-out replaces it
2	Health index version 1 stays the default; version 2 is exposed conditionally
3	Exact Shapley attribution for the rule scores; SHAP only where it is the right tool
4	The dashboard renders only computed values
5	The gate is the product, not the model
6	Transfer targets are screened on commensurability, not availability
7	The target was the binding constraint, not the model
8	Model capacity buys within-protocol skill and spends it on transfer
9	The protocol-shift collapse replicates on CALCE, but its shape is dataset-specific
10	The leave-one-cohort-out estimate is noisier than the effect it is used to measure
11	Unsupervised detection finds structure, not harm
12	An absolute SOH target on CALCE recovers four cells and refutes the claim that learned models add enormously
13	The estimator-variance result survives the target correction as a ratio
14	Rated capacity is declared, not defaulted

3.18 ALGORITHM SUMMARY

The estimators of the previous sections combine into one procedure per cell, summarised below in the order in which the steps run. Every step that can fail ends in a refusal with a stated cause and not in a default.

```

for each cell in the capture:
  discharges <- segment(stream)           # by current, time gaps
  if capture-level gate fails:  refuse(cause)
  for each discharge n:
    R(n)  <- trailing median of last 11 load-step estimates
    if no R yet:  mark n "unmeasured"; continue

```

```

window <- [V_low, V_high] shifted by |I_n| * R(n)
if window not spanned: mark n "unmeasured"
if truncated / implausible / reference too short: refuse n
Q(n) <- charge delivered inside window (coulomb count)
SOH(n) <- Q(n) / Q_ref
if enough complete discharges (30):
    RUL <- cycles until fade of complete-discharge capacity
    crosses the EOL threshold (90 % default)
else: refuse RUL
if evidence inconsistent: confidence <- low
return SOH, R, RUL, confidence, band, refusals

```

Remaining life is taken from the capacity of complete discharges only: extrapolating the windowed health of partial discharges was tested and is not accurate enough to report (Chapter 5). The reference Q_{ref} is the declared rated capacity when the source gives one, or the cell's own early capacity otherwise, and the source of the reference is returned with the result. The error band attached to each output is the measured error of that output on the validation data and is not a quantity computed from the cell being scored.

3.19 VALIDATION PROTOCOL

Ground Truth. Ground truth is the cyclers's measured discharge capacity, which never enters an estimate. SOH truth is capacity over the median of the cell's first five cycles. For partial-cycling cells, whose checks are about 100 cycles apart, the partial-discharge study scores each cell against its first check, and the remaining-life study uses the capacity over the checks within the first ten cycles. Each is a reference a BMS can hold. A reference drawn from the cell's whole record uses the future, and Section 5.5 quantifies the resulting inflation.

Pre-Declared Configurations. Primary windows, comparison arms, tolerances and scoring were fixed before each study ran. Choices made after seeing a result are labelled exploratory.

Benchmark. The benchmark uses leave-one-cell-out (LOBO, 31 folds) and leave-one-cohort-out (LOCO, 9 folds) validation. R^2 is computed against the training-fold mean, with 95% bootstrap intervals resampling folds (2,000 resamples). Hyperparameters were fixed and not tuned per fold, except ElasticNet's penalty, chosen by inner cross-validation within the training fold.

Aggregation. Errors are summarised per cell, then across cells, so long-lived cells do not dominate.

3.20 PARAMETERS OF THE ESTIMATORS

The estimators have few parameters, and each was fixed on the development data before the external datasets were scored. Table 3.7 lists those quoted in this chapter. None was changed when the estimators were applied to the Oxford and NASA data.

Table 3.7: Parameters Fixed Before External Validation

Parameter	Value	Used for
Maximum gap between rest and load samples	60 s	Accepting a load-step estimate
Minimum current change at the step	0.05 A	Accepting a load-step estimate
Length of trailing resistance median	11 steps	Ohmic correction
Rest threshold of mode inference	± 0.02 A	Schema-level operating mode
Serial sampling period (rig)	1 s	Rig firmware
Default end-of-life threshold	90%	RUL target (80% is the convention but is seldom reached on CALCE)

Fixing the parameters in advance has a cost. They are not the best possible values for any one dataset, and the errors reported on the external data are therefore a fair statement of what the estimators do on data they were not tuned on, and not the best that could be obtained by tuning.

3.21 EVALUATION METRICS

- **SOH error:** per-cell mean absolute error in SOH (0.01 equals one percentage point), the median over cells, a 95% interval that resamples cells, and the worst cell.
- **Coverage:** cells measured out of cells scored. Refusals count against coverage and are reported with their cause.
- **Resistance:** Spearman correlation against the cyclers’ resistance column over each cell’s life, direction of growth, and growth ratio.
- **RUL:** the share of estimates within ± 20 cycles, the median error, and the median bias, by true distance to end of life and by predicted distance.
- **Sufficiency:** median and 90th-percentile error by consistency class, and the correlation between u and absolute error.
- **Benchmark:** LOBO and LOCO R^2 against the training mean with bootstrap intervals, and LOCO mean absolute error.

3.22 EXPERIMENTAL CONTROLS AND REPRODUCIBILITY

Every reported number is regenerated by a named script from committed artifacts (Appendices A and B), and a test (`test_reported_numbers`) ties each figure quoted in the documentation to the artifact that produced it, so a re-run experiment with a stale paragraph fails the build. A control shuffles each cell's capacity in time to confirm that the benchmark harness does not manufacture skill. A continuous-integration job reproduces the headline benchmark end to end.

CHAPTER 4

IMPLEMENTATION AND DEPLOYMENT

4.1 IMPLEMENTATION OVERVIEW

The previous chapter defined the design: one schema, one scoring function, two estimators that fit nothing across cells, and explicit refusals. This chapter describes how that design was realised. The implementation has three parts:

- **Part A – Telemetry and estimation software:** the packages that acquire, validate, segment and score telemetry (Sections 4.2 to 4.8).
- **Part B – Hardware bench rig:** the instrumentation design, firmware and physical bring-up that supply real telemetry to the software (Section 4.9).
- **Part C – Service tier and deployment:** the REST API, the gateway, the dashboard, testing, continuous integration and containers (Sections 4.10 to 4.17).

4.2 DEVELOPMENT ENVIRONMENT AND TOOLS

The system is written in Python, Node (JavaScript), React and C++ (Arduino firmware), with about 23,000 lines of Python in the core package and 44 test modules. The components that matter for reproducibility are listed in Table 4.1.

4.3 CODE ORGANISATION

The core package `src/bms` is divided into packages with one responsibility each (Table 4.2).

Outside the package, the repository holds the following.

- `mern/`: the Express gateway and React client.
- `firmware/beacon_rig/`: the Arduino sketch.
- `scripts/`: the study and demo scripts.
- `tests/`: the test modules.
- `docs/adr/`: fourteen architecture decision records, including those that record reversed decisions.

Table 4.1: Software Components Used

Software	Role
Python (≥ 3.10)	Core language; CI tests 3.10 (the declared floor) and 3.12
pandas, NumPy, scikit-learn	Telemetry frames, numerical work, benchmark estimators and bootstrap analysis
XGBoost, PyTorch (optional)	Benchmark methods (gradient boosting, LSTM)
FastAPI, uvicorn	Scoring, telemetry ingest and fleet store as a REST service
cantools, python-can (optional)	DBC decoding and CAN capture; CI also runs the suite with them uninstalled
pyserial (optional)	Reading the physical rig's serial port
Express (Node)	Gateway between the client and the Python service
React, Vite	Dashboard client
Arduino C++ / <code>arduino-cli</code>	Rig firmware, compiled for ESP32, ESP8266 and AVR
pytest, ruff, mypy	Tests, linting and type checking
Docker, Docker Compose	Multi-stage image and three-service stack
GitHub Actions	Eight CI jobs, including a reproduction of the headline study

4.4 TELEMETRY ACQUISITION AND THE UNIFIED FRAME

The three transports converge on one unified telemetry frame with the columns `cell_id`, `test_time_s`, `voltage_v`, `current_a`, `temperature_c` and `soc`.

- **CAN bus.** Frames are decoded against a DBC definition, and a coverage gate refuses a capture whose required signals are missing.
- **USB serial.** A line source feeds a parser that classifies each line, verifies its checksum and validates every field against a declared range (Section 4.7).
- **Research datasets.** Loaders read the NASA MATLAB files and the CALCE and Oxford records into the same frame.

4.4.1 Interface Between Acquisition and Scoring

A serial source is a protocol with one method, `lines()`. A physical port, a recorded text file, a list in a test and the deterministic emulator are interchangeable, so the demo path and the hardware path differ in one constructor call. The emulator deliberately generates *wire-format text* and not record objects. Emitting objects would have been less code but would have proved nothing, because the parser, the checksum, the schema validation and the coverage gate would all be bypassed in exactly the path the tests exercise. The emulator even prints an ESP32 boot banner, so the sentinel filter runs on

Table 4.2: Organisation of the Core Package

Package	Responsibility
telemetry	CAN capture and replay, serial ingestion and protocol, cycle segmentation, and the shared scoring function
io	Dataset loaders (NASA, CALCE, Oxford) and DBC examples; the unified schema is in preprocessing
features, preprocessing	Behaviour features and data conditioning
risk, health	Rule-based risk and health index (heuristic layer); measured field SOH, voltage-window, evidence and error-band estimators
rul	Fade-extrapolation remaining useful life, and the earlier heuristic estimate
guardian, digital_twin	Human-readable diagnosis and the state-machine twin
explain	Exact Shapley attribution over the rule scores
uncertainty	Distribution-free uncertainty (conformal prediction) and coverage under shift
benchmarks	Published methods run through the project’s own validation gate
adaptive	Governed model promotion across datasets and protocols
detect, physics	Unsupervised usage segmentation; physics-grounded degradation models
simulation	Synthetic telemetry for tests and the no-hardware demo
api, dashboard	FastAPI service and the static HTML dashboard

every test. Recording composes at the same seam: one decorator makes a live port, a replay or the emulator recordable, and the file on disk is exactly the bytes that produced the result.

4.5 UNIFIED SCHEMA IMPLEMENTATION

Each source has its own loader, whose only job is to map that source onto the schema of Section 3.6. The NASA and CALCE loaders read the laboratory files, the CAN loader decodes frames, and the serial loader parses lines. Every loader corrects the current sign convention of its source before the validator runs, so the pipeline never sees a mixed convention. The derived fields, power, operating mode and state-of-charge band, are recomputed after each preprocessing run and not stored from an earlier one, so a change to a threshold takes effect everywhere at once. A loader that cannot supply a required channel returns a frame that the validator rejects with the name of the missing column, which is preferable to a loader that fills the gap with a plausible constant.

4.6 CAN AND DBC ADAPTER

The CAN adapter decodes raw frames described by a DBC file into the unified schema. It uses an established library, `cantools`, for the decoding and does not implement the bit extraction itself. This is a deliberate choice: the big-endian bit numbering used in DBC files is a well-known source of subtle errors, and an almost-correct hand-written decoder would produce values that look plausible and are wrong, the failure the project is designed to avoid.

The decoder is general, because it parses any standard DBC file. The bundled example describes the primary battery status message of a Renault Twizy, identifier 0x155, transcribed from the public documentation of the Open Vehicles Monitoring System. Its correctness is checked against an independent source: a test decodes the exact bytes of the worked example in that documentation and asserts that the current and state of charge match the published result, 58.25 A and 69.98%, so the test does not merely confirm that the code agrees with itself.

4.6.1 Vehicle Registry

The single-file adapter is generalised by a registry in which several vehicles' DBC files can be registered. Given a batch of raw frames, the registry identifies the vehicle by comparing the observed identifiers with the identifiers each vehicle defines, and decodes accordingly. If no registered vehicle matches with confidence, it raises an error and does not choose the closest match, which is the same refusal stance as elsewhere in the system. By default exactly one real vehicle is registered. A synthetic fixture, clearly named as not a real vehicle, can be added in tests to exercise the disambiguation logic and is never part of the default registry, so application code cannot mistake it for real coverage.

4.6.2 Limits of the Adapter

The adapter proves the pattern, a CAN frame becomes decoded signals and then a unified frame, against real vehicle protocol data. It does not prove broad coverage of manufacturers. Most vehicles supported by community projects are decoded by hand-written code and not by a portable DBC file, and manufacturer files are proprietary. The documented message carries current and state of charge but no voltage or temperature, so the validator correctly reports that it cannot be scored on its own. Reading a live bus is implemented as a separate source built on the `python-can` library, layered under the decoder and exposed through a live-capture route, but it has not been exercised on a physical vehicle.

4.7 SERIAL WIRE PROTOCOL

Each frame is a line of text with a sentinel, a record kind, a JSON body and an optional checksum (Fig. 4.1). The record kinds are HELLO (the rig declares its channels and

rated capacity), D (data) and S (status).

```
BEACON1 D {"t":12.5,"v":3.91,"i":-1.48,"tc":27.4,"soc":82.1}*06
```

Figure 4.1: A data frame: sentinel BEACON1, kind D, JSON body, and the XOR checksum *06, ending in a newline.

4.7.1 Frame Delimitation

Framing is by sentinel and not by length. Every microcontroller prints boot banners and debugging text, so a length-prefixed parser would resynchronise after such noise only by luck. A sentinel parser discards anything not beginning with BEACON1 as counted noise and continues, which makes the protocol self-synchronising: it recovers at the next newline after any corruption, with a loss of at most one frame.

4.7.2 Error Detection

The checksum is an 8-bit XOR over the body (a longitudinal redundancy check). It detects all single-bit errors, but it cannot detect an even number of bit errors in the same bit position, any reordering of bytes, or corruption of the sentinel and kind fields, which lie outside the body. Under a random-corruption model, 0.39% of corrupted frames would pass. A CRC-8 occupies the same eight bits and detects all bursts up to eight bits and byte transpositions, so it is recorded as the protocol's clearest improvement. It was not adopted for the first bring-up because the XOR form is three lines of C and legible in a serial monitor.

4.7.3 Channel Utilisation

Table 4.3 gives the measured frame sizes and the derived link performance. At 1 Hz the link is over-provisioned by more than two orders of magnitude, so the verbose JSON codec is the default: it costs 39% more bandwidth than the compact codec and buys human-legible frames during bring-up.

Table 4.3: Frame Size and Link Performance of the JSON Codec

Quantity	Value
Data frame, mean size	72.4 B
Bits on the wire per frame	724
Maximum sustainable rate at 115200 baud	159 frames/s
Utilisation at 1 Hz, 115200 baud	0.63%
Utilisation at 1 Hz, 9600 baud (as built)	about 7.5%

4.8 IMPLEMENTATION OF REFUSAL GATES

A request through the serial path passes the following stages. The route validates the request, a line source yields lines (decode errors become counted rejections, not

exceptions), the parser classifies each line and verifies its checksum, the serial pipeline applies the gates below and assembles the unified frame, and the shared scoring function runs. A decode-statistics record rides along the whole way so that a capture that silently dropped most of its lines cannot look like a clean one. Every gate returns a result carrying a refusal and not an exception (Table 4.4).

Table 4.4: Refusal Gates and Their Purpose

Gate	What it prevents
Missing required channel	A comparison against <code>NaN</code> is false, so an absent temperature reads as “not hot” on every row. This defect shipped once and is why the gate exists.
Unimplemented schema id	Identical field names meaning different things across a schema change.
SOC given as a 0–1 fraction	A value that passes the range check but means the wrong thing. Caught per capture, because a fraction lies inside the 0–100 range.
Reversed current sign	A rig reporting discharge as positive is flagged as a likely cause; the run is refused because no complete discharge is found, although the stream looks perfect.
Time running backwards	A brown-out restarts <code>millis()</code> at zero, and coulomb counting would cancel the second session against the first. Sorting would hide it.
Fewer than 50% of records parsed	A wrong baud rate or mismatched firmware, as distinct from cable noise.
No rated capacity declared	C-rate is current divided by rated capacity, and an assumed 2.0 Ah on a 3.4 Ah cell reads 1.7 C while drawing 1 C.
No complete discharge cycle	Capacity is comparable only across equal depths of discharge, so partial cycles are excluded and not scaled.
Nothing passed the promotion gate	The adaptive calibrator refuses to predict fade, and no transport routes around this.

Two properties of this design are load-bearing. First, range checks *reject* rather than clamp: an out-of-range value is evidence that the reading is not what the schema says it is (a disconnected thermistor floats, an unpowered INA219 reads zero), and clamping would produce a value that looks measured and is not. Second, some errors are not expressible per record, so the capture-level checks sit one level above the record-level ones. Refusing is also not the same as withholding everything: segmentation and coulomb counting do not divide by capacity, so the capture is measured and reported and only the behaviour scoring is withheld. The capacity gate was moved to the scoring stage after the first implementation discarded valid cycle measurements.

4.9 HARDWARE BENCH RIG

4.9.1 Requirement and Design Calculations

The rig must measure one lithium-ion cell during charge and discharge and supply the five schema channels (terminal voltage, signed current, temperature, state of charge and elapsed time). State of charge is derived, not measured. The binding accuracy requirement is on capacity, with an acceptance criterion of within 5% of an independent reference.

The hardware design document works the instrumentation from datasheet figures; none of it has been measured on assembled hardware, and the calculations below are therefore design values. Current is measured high-side through a shunt read by an INA219, so that the terminal-voltage measurement is not shifted by the shunt drop. The shunt sizing is a trade between range, resolution and self-heating (Table 4.5).

Table 4.5: Shunt Sizing for the INA219 (320 mV Full Scale, 10 μ V LSB)

R_s	$I_{\max}$	I_{LSB}	Power at 2 A	Verdict
0.1 Ω	3.2 A	100 μ A	0.40 W	Selected for ≤ 3 A
0.05 Ω	6.4 A	200 μ A	0.20 W	1C on a 3.4 Ah cell
0.01 Ω	32 A	1.00 mA	0.04 W	Wasteful here

Three consequences are recorded. Discharge must stay below 3.2 A with the 0.1 Ω shunt, because saturation is not a graceful failure: the reading clips, coulomb counting under-integrates and capacity reads low with nothing to indicate why. Self-heating shifts the shunt value, and a shunt reading high makes current read low, a systematic capacity error in one direction. The shunt drop is also subtracted from the voltage the INA219 reports on its load side, which is 200 mV (5% of a 4 V reading) at 2 A, so the bus voltage must be wired to the cell side of the shunt or corrected in firmware.

The error budget carries the component errors through to capacity. Combining the INA219 gain error (about 0.5%), the shunt tolerance (1%) and shunt self-heating (about 0.2%) in quadrature gives about 1.14% of reading, and because capacity is the integral of current, a gain error on current is a gain error on capacity. The expected capacity accuracy is about 1.2%, dominated by the shunt tolerance, against the 5% acceptance criterion. A 0.1% shunt would roughly halve it. Offset and timing contribute negligibly. These are datasheet-based design figures, and measurement against a reference meter has not been done.

4.9.2 Implemented Rig Configuration

The rig that was physically brought up differs from the design document in two components, and that difference is stated here and not hidden (Table 4.6, Fig. 4.2).

Protection must be in hardware in series with the cell, never in the monitoring firmware, because a firmware hang must not leave the cell unprotected. The monitoring

Table 4.6: Design Specification Versus Implemented Rig

Element	Design document	As built
Microcontroller	ESP32-WROOM-32 DevKit	NodeMCU ESP8266 (CH340 USB bridge)
Current and voltage	INA219 on I ² C	INA219
Temperature	DS18B20 (1-Wire)	LM35
Cell	Protected 18650	HONGLI ICR-18650, single cell
Serial rate	115200 baud	9600 baud (CH340 bridge limitation)

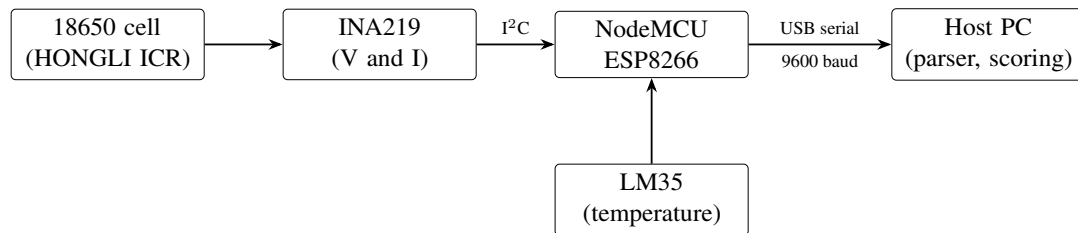


Figure 4.2: Implemented bench rig: measurement chain from the cell to the host.

electronics are powered from USB, not from the cell. The design power budget is about 48 mA typical against 500 mA available from USB, and the firmware keeps the radios disabled, not to save power but because transmit current spikes can brown out a weakly supplied board, which would restart the time base.

4.9.3 Firmware

The firmware is one board-agnostic Arduino sketch. The sensors sit behind four replaceable functions (`readVoltageV`, `readCurrentA`, `readTemperatureC` and `readSocPercent`), so a board with only some sensors can still run. At boot it probes each sensor, emits a status record naming what failed, and declares in its `HELLO` only the channels it can actually supply, together with the rated capacity. The host then scores the channels present and refuses the rest by name. The sketch compiles for three targets on every push (Table 4.7); the continuous-integration build is the stub configuration without the sensor libraries, and a test compares its declared field set, as a set, against the schema.

Table 4.7: Firmware Compilation Record (`arduino-cli`, all warnings enabled)

Target	Flash used	RAM used
Arduino Uno (AVR)	8,990 B of 32,256 (27%)	460 B of 2,048 (22%)
NodeMCU v2 (ESP8266)	240,340 B of 1,048,576 (22%)	28,372 B of 80,192 (35%)
ESP32 Dev Module	272,324 B of 1,310,720 (20%)	22,140 B of 327,680 (6%)

4.9.4 Bring-Up Procedure

Bring-up was done in two stages that must not be combined. Stage A proves the transport with nothing at risk: no battery and no sensors. Stage B adds one cell and the sensors, replacing the four read functions one at a time. Doing both at once would make a wiring fault and a protocol fault indistinguishable with a lithium cell connected. Stage A passes when the accepted fraction is at least 0.99, a `HELLO` was seen, there are no time reversals, and the C-rate basis names the capacity the sketch declared, and when replaying the saved capture reproduces the live numbers.

A bare board can produce a full report: the stub firmware’s synthetic profile scores as complete cycles, so a rig with no sensors attached yields a health index and a remaining life that look like success. The records are well formed and in range, so the software cannot detect this, and the only defence is procedural: any hardware claim must state which of the four read functions were real. The captured file from a board with no sensors is kept in the repository as a labelled counter-example.

4.10 SERVICE TIER: REST API, GATEWAY AND DASHBOARD

The service tier has three components (Table 4.8). The gateway has exactly one job, translating HTTP calls from the client into calls to the Python service, and every route in it is a thin forward, so a wrong behaviour is fixed in the Python service and not in the gateway.

Table 4.8: Services

Service	Stack	Responsibility
api (port 8000)	FastAPI, uvicorn	Scoring, telemetry ingest, fleet store
gateway (port 5000)	Express	Client-facing aggregation
client (port 5173)	React, Vite	Fleet table, detail, twin, replay

The principal endpoints are listed below, together with the telemetry routes for CAN replay, serial replay, coverage, the twin and the thermal view.

- `GET /healthz`: service liveness.
- `POST /pipeline/simulate`: run the pipeline on a simulated input.
- `GET /batteries` and `GET /batteries/{id}`: the battery list and one battery.
- `GET /batteries/{id}/timeline`: the per-battery timeline.

The `/healthz` endpoint is explicitly service liveness and not battery health: in a battery-monitoring system the naming collision is a real hazard and is called out at its definition. The container health check polls it.

The React client is a single-surface dashboard. It shows the fleet of batteries with health, risk and remaining life for each, a battery detail view, a bench-rig panel, a digital-twin sheet, and a banner stating where the run's data came from. An earlier version had separate tabs for fleet, CAN replay, thermal and transfer views. They were removed because two parallel dashboards meant two provenance banners that could disagree about the same run. The twin sheet is present but displays “not available” for the observed and predicted state, because the twin module does not yet emit its state into the dashboard payload, and no substitute trajectory is drawn. A Guardian explanation is shown next to the run it explains, and refusals are rendered as results and not as errors.

Fig. 4.3 and Fig. 4.4 show the dashboard. Both are screenshots of the running system, taken on a synthetic fleet, which the interface labels as simulated data; the bench-rig panel below the fleet summary is the only part backed by a physical measurement.

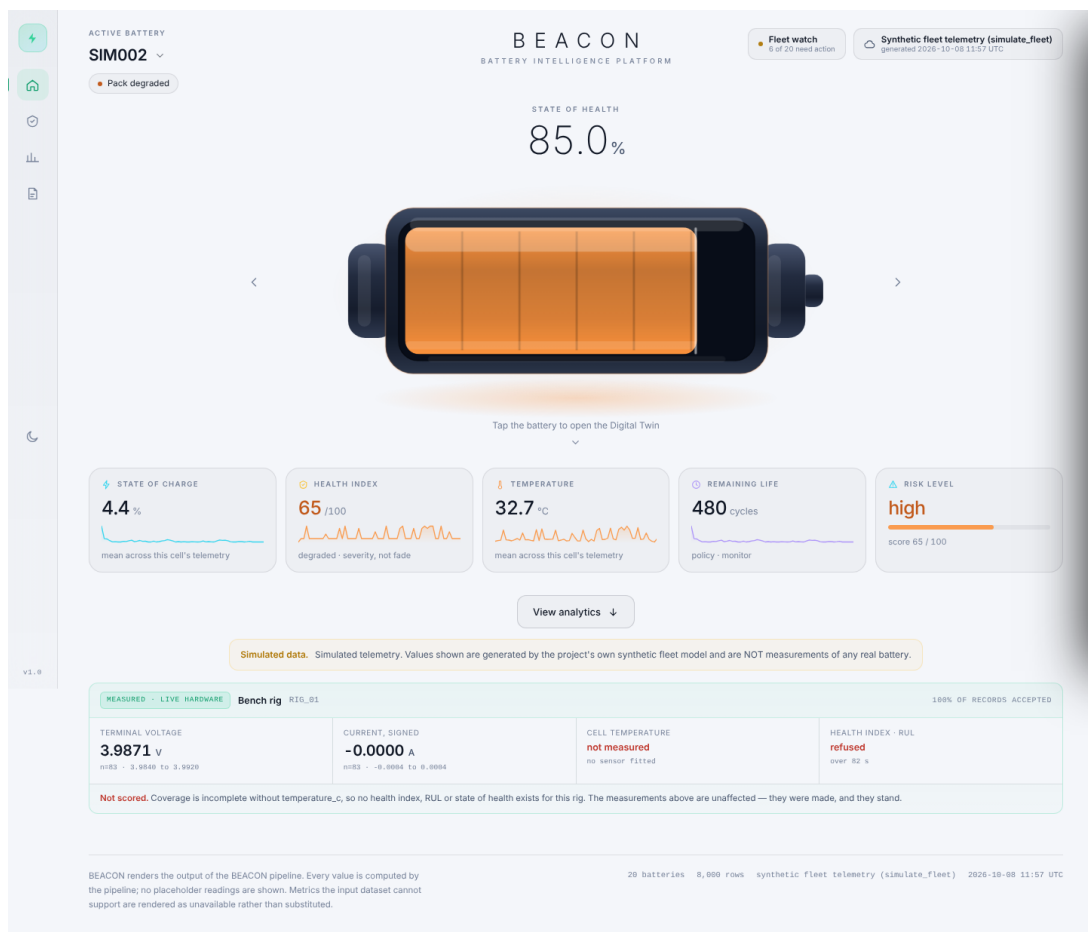


Figure 4.3: Overview screen of the BEACON dashboard (light theme) on a synthetic fleet. The banner marks the values as simulated, and the panel at the bottom shows the measured bench-rig readings, with the health index and remaining life refused because no temperature channel exists.

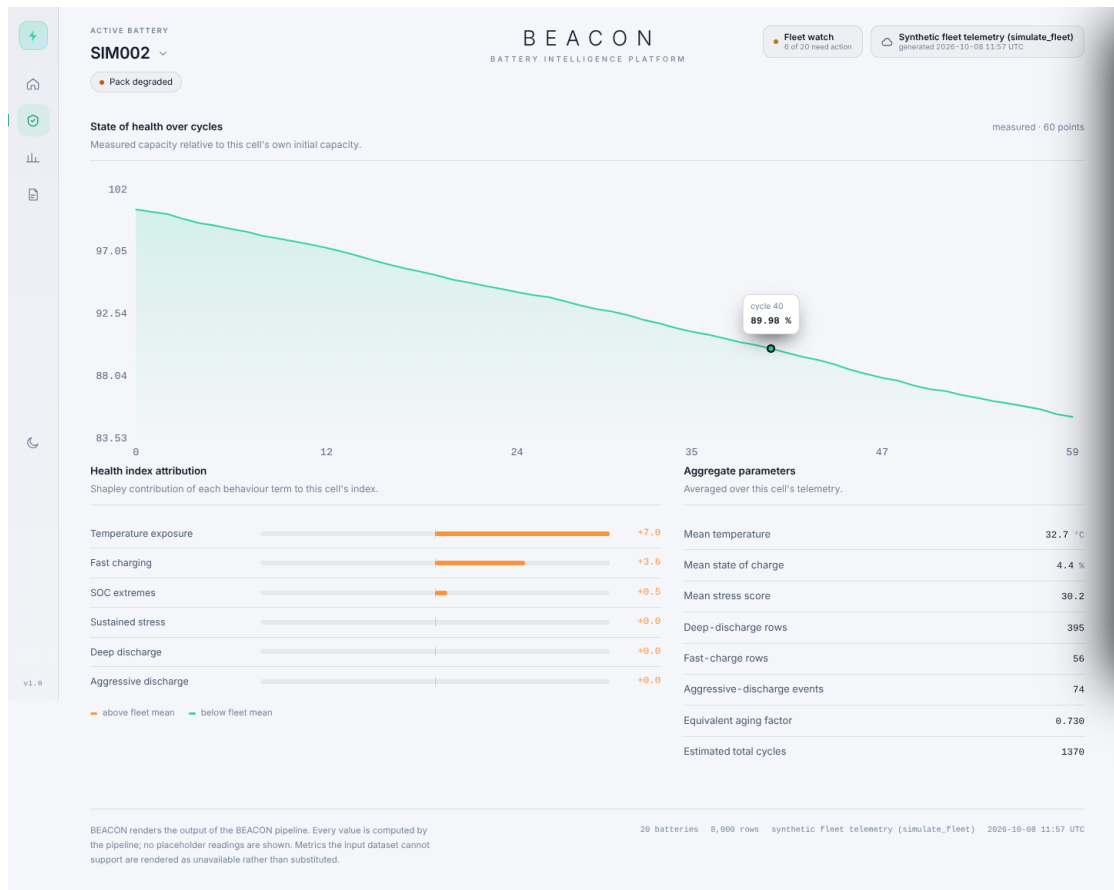


Figure 4.4: Battery health screen (light theme): state of health over cycles for one battery, the attribution of its health index to behaviour terms, and its aggregate usage parameters (synthetic fleet).

There are two deliberate absences. There is no database: the fleet store is in memory, and adding a database to the compose file would imply a persistence layer that does not exist, so the compose file says so. Persistence is the next substantial piece of work.

4.11 DIGITAL TWIN AND FLEET STORE

The twin is implemented as a domain module with no transport dependency. The API layer is a thin adapter over it, so adding another front end, for example a message broker, means writing a new adapter and leaves the module unchanged. The fleet store holds the latest twin state, the transition history and the telemetry of the most recent run for each battery. It is held in process memory and does not survive a restart, which is stated in the API documentation and shown in the compose file by the absence of a database service.

The twin also carries a field called evidence confidence, which takes one of four values: validated, heuristic, mixed or not applicable. It states whether the causes the Guardian identified for a battery are supported by the calibration work or are hand-chosen thresholds. Temperature exposure is the only cause with a measured, cohort-controlled effect (0.0038 Ah per °C), while the current-based causes and the health index itself are

marked heuristic. This does not change what the Guardian detects. It changes what the system says about its own certainty, which was previously not visible in the output.

4.12 API REFERENCE

Table 4.9 lists the principal routes of the Python service. Further routes cover live CAN capture, serial emulation, live serial capture, latest and live state, thermal and twin views, transfer feasibility and the dataset listing. The interactive documentation generated by FastAPI from the same code is served at `/docs`, so it cannot drift from the code in the way a hand-written reference can.

Table 4.9: Principal Routes of the Python Service

Route	Purpose
GET <code>/healthz</code>	Service liveness, with run and battery counts
POST <code>/pipeline/simulate</code>	Score a simulated fleet and fold it into the store
GET <code>/batteries</code>	Twin state of every battery seen
GET <code>/batteries/{id}</code>	Twin, Guardian report, risk, history, evidence confidence
GET <code>/batteries/{id}/timeline</code>	Per-cycle stress, state of charge and temperature
POST <code>/telemetry/replay</code>	Replay a recorded CAN log through the pipeline
POST <code>/telemetry/serial/replay</code>	Replay a recorded serial capture
GET <code>/telemetry/coverage</code>	Report which required signals a DBC file provides

A battery that is in the store but absent from the most recent run has a twin state and a history but no timeline, and a request for its timeline returns a not-found response and not a stale series. Every request body is validated by Pydantic. The duration of a live capture is bounded above by 300 s, the number of emulated cycles lies between 1 and 50, the sampling period lies between 0 and 3600 s and the accepted fraction lies between 0 and 1, so no request can start a capture that never returns.

4.13 SECURITY CONSIDERATIONS

The service is a local bench and analysis tool, and its security review asked a narrow question: can a caller who reaches the service make it do something it should not? The review was carried out against the repository and recorded, with the findings and the decision taken on each. Table 4.10 summarises them.

The one substantive finding was that three routes opened a filesystem path taken from the request body without any containment. A caller could name a private key or

Table 4.10: Security Review of the Service

Finding	Outcome
Request-supplied file paths on replay and coverage routes	Fixed: path must resolve under an allowed root
Cross-origin policy permissive by default	Made configurable; not presented as a security boundary
Secrets and language-model integration	Verified absent by search of the repository
Request validation	Adequate; bounds on durations and counts
Serial parser under adversarial input	Safe: bad lines counted, values rejected, capture refused below a threshold
Notebook metadata	Noted; left to the author

a configuration file, and the per-line rejection reasons returned by the serial path could carry fragments of what was read. The difference between a not-found and a parse error also revealed whether a file existed. The fix resolves the path first and requires it to lie under an allowed root, and rejects it with a client error before any existence check, so an out-of-bounds path cannot be used to probe the disk. A list of allowed roots was chosen over a blocklist of path patterns, because blocklists on path strings can be defeated by symbolic links, percent-encoding and short file names, whereas resolving first and then asking where the path ended up is decided after any such trick has been applied. The allowed roots can be replaced through an environment variable, and the variable replaces the defaults and does not extend them, so setting it cannot leave a development convenience in place. Thirteen tests cover the containment.

4.13.1 Threats Outside the Scope of the Design

The service has no authentication, no rate limiting, no transport security and no tenant isolation, and it keeps one global in-memory store. These are consequences of its scope as a local tool and are recorded as such. The correct remedy for a shared deployment is an authenticating reverse proxy in front of the service and not code inside it.

4.14 DASHBOARD COMPONENTS

The dashboard is a React application with one surface, rendered from the pipeline output. Table 4.11 lists its views. (The repository also keeps earlier per-panel components that the current surface no longer uses.) The panels that display a measured result carry the provenance of the value, so a reader can tell whether a figure comes from a vehicle, a rig, a dataset or a simulation.

The design mock-up of the dashboard contained placeholder readings, such as a state of health of 92.4% and a remaining life of 1,245 cycles. A decision record fixes that the interface renders only values that were computed, and shows a refusal or an empty state in place of a placeholder when the pipeline has produced nothing. This is the same stance as in the estimators, applied to the display.

Table 4.11: Views of the Dashboard

View	Content
Fleet risk distribution	Risk category of every battery
Behaviour flag incidence	How often each flagged behaviour occurs
Battery health	Stress score over cycles, risk and health-index attribution
Aggregate parameters	Usage parameters of the selected battery
Guardian explanation	Plain-language report of the causes found
Bench rig panel	Measured readings, with refusals shown where a channel is missing
Twin sheet	Observed and predicted state; shows not available until the twin is wired in
Provenance banner	Source of the displayed values

4.15 HEALTH REPORT CARD

The health report script renders the outputs as a plain-language card for one battery, in five parts: how healthy it is, how long it has left, what the usage is doing to it, what to do, and what the report cannot tell. Every figure carries its validation error band, which is taken from a file of measured bands and pinned by a test, and each output is labelled measured, estimate or heuristic. When ground truth exists, the card is followed by a check against the laboratory, which the card itself was not shown. Section 5.11 reports a worked example.

4.16 TESTING, CONTINUOUS INTEGRATION AND REPRODUCIBILITY

There are 44 test modules, and three kinds of test are not the usual kind.

- **Prose-to-evidence pinning.** A test ties every figure quoted in the documentation to the artifact that produced it, so re-running an experiment and forgetting to update a paragraph fails the build. It caught a stale figure that had already been published.
- **Cross-artifact structural tests.** The firmware's declared field set is compared as a set against the schema, and the documented schema table must equal the rendered one. Substring assertions had already let a defect through: a renamed field left the schema id untouched, so the sketch kept announcing a version it no longer spoke.
- **Known-defect pins.** Some tests assert behaviour that is not desirable, with a comment saying so, for example that the risk score is too coarse to see a sevenfold change in average stress. Changing it would silently re-score every published figure, so it is pinned and a future change has to be deliberate.

Continuous integration runs eight jobs: lint and type checking, the test suite on Python 3.10 and 3.12, a job that uninstalls the optional CAN dependency and re-runs the

suite, a job that reproduces the headline benchmark end to end, a firmware compile for three targets, the gateway tests, the client build, and the Docker build. The job without the optional dependency exists because “it is optional” is otherwise an untested claim. The Python-version matrix exists because testing only the newest version lets a declared floor rot silently.

4.16.1 Test Modules by Subsystem

Table 4.12 lists some of the test modules and the behaviour each one pins. Each subsystem has tests at its own boundary, so a fault is caught where it is introduced and not where it is first noticed.

Table 4.12: Selected Test Modules and Verified Behaviour

Module	Behaviour checked
test_api	Route behaviour, repeated identical seeded run produces no new twin transitions
test_api_path_containment	Absolute sensitive paths, parent-directory traversal and empty paths are rejected before any existence check; the environment-variable override is covered
test_can_dbc	Decoded values match the published worked example; missing voltage and temperature reported by the validator
test_can_vehicle_registry	Vehicle identified from observed identifiers; ambiguous input refused
test_digital_twin	State derivation and transition recording
test_serial_telemetry	Framing, checksum, rejected values, capture-level refusal
test_hardware_readiness	Truncated lines, corrupted checksums, non-finite and out-of-range fields
Gateway tests (gateway, telemetry)	Each route forwards to the Python service and returns its result unchanged

4.17 CONTAINERISATION AND DEPLOYMENT

The Docker image is multi-stage on a slim Python 3.12 base, with the compiler confined to the build stage, a non-root user, and a health check that polls service liveness. Docker Compose brings up the API, the gateway and the client, with the gateway held until the API reports healthy, and a separate study service writes the result CSVs back to the host. The system runs on a fresh clone with no hardware, no database, no API keys and no dataset download:

```
python main.py                # end-to-end run
python scripts/run_serial_demo.py # emulated rig
python -m pytest tests/ -q      # the test suite
docker compose up              # API, gateway, client
```

4.18 OPERATIONAL WORKFLOW

A typical session with the system follows the same path whatever the source. An operator starts the three services, or runs the single command that produces the end-to-end result, and selects a source: a simulated fleet, a recorded CAN log with its DBC file, a serial capture, or a live capture from the rig. The service converts the source to the unified frame, applies the schema and capture-level checks, and scores the result. The dashboard then shows the fleet table first, with each battery's health, risk and remaining life, and each value labelled with its source. Selecting a battery opens its detail, where the health over cycles, the attribution of the health index and the twin history are shown. If a capture was refused, the dashboard shows the refusal and its cause in the place where the number would have been.

The same path serves a bench session. With the rig connected, the operator records a capture to a file and replays it through the serial route, which gives the same result as scoring it live and leaves a file that can be re-scored after any change to the code. Replay from a recorded file is also how the results of this report were reproduced, since a file scored twice gives the same output, and a live cell does not.

4.19 CHAPTER SUMMARY

This chapter described the realisation of the design: a shared scoring path fed by three transports through a narrow seam, a self-synchronising wire protocol with refusal gates at both record and capture level, a physical rig brought up in two stages, a three-service stack served from containers, and a test and CI strategy aimed at keeping the documentation and the code honest. What this implementation *achieved* is reported in the next chapter.

CHAPTER 5

RESULTS AND DISCUSSION

This chapter reports what the system measured and interprets each result where it appears. Every number is traced to a committed artifact (Appendices A and B). Errors are mean absolute errors in SOH (0.01 equals one percentage point) per cell, summarised as a median over cells with a 95% bootstrap interval that resamples cells, unless stated otherwise. Results that failed are reported alongside those that worked.

5.1 PERFORMANCE OF FITTED MODELS UNDER PROTOCOL SHIFT

On the NASA benchmark (31 cells, nine cohorts; the isotonic signal ceiling for the target SOH is 0.569), eleven of the twelve methods lost skill, and the twelfth, a train-mean baseline, cannot lose it by construction, from leave-one-cell-out (LOBO) to leave-one-cohort-out (LOCO) validation (Table 5.1). LOBO rank carried almost no information about LOCO rank (Spearman $\rho = +0.084$, $p = 0.795$, $n = 12$). Across fourteen scored methods, the spread of LOCO point estimates (2.06 in R^2) was smaller than the width of the best method’s own interval (3.66).

Table 5.1: Benchmark on 31 NASA Cells in Nine Cohorts (Six of the Methods Shown); LOCO Intervals Are 95% Bootstrap Over Folds

Method	LOBO R^2	LOCO R^2	LOCO 95% CI	LOCO MAE
XGBoost	0.732	0.459	[−2.80, 0.86]	7.62%
Age-linear baseline	0.483	0.406	[−0.43, 0.51]	10.71%
ElasticNet	0.609	0.355	[−1.65, 0.76]	10.31%
GPR (Matérn)	0.725	0.207	[−3.72, 0.43]	17.06%
Random forest	0.773	0.150	[−1.09, 0.90]	10.67%
LSTM (10-cycle windows)	0.593	−0.185	[−2.09, 0.66]	17.36%

Fig. 5.1 shows the same comparison for all fourteen scored methods: nearly every bar drops from the blue (LOBO) to the red (LOCO) value when the held-out unit changes from a cell to a whole protocol cohort.

An ablation isolates where the skill comes from (Table 5.2). XGBoost on cycle number alone scored a LOCO R^2 of -0.27 , and on behaviour features alone -0.29 . Only their combination reached 0.459, while a straight line on cycle number reached 0.406. Shuffling each cell’s capacity in time placed every LOCO interval at or below zero, so the harness does not manufacture skill. Usage features therefore added about 0.05 in R^2 over counting cycles, within the intervals.

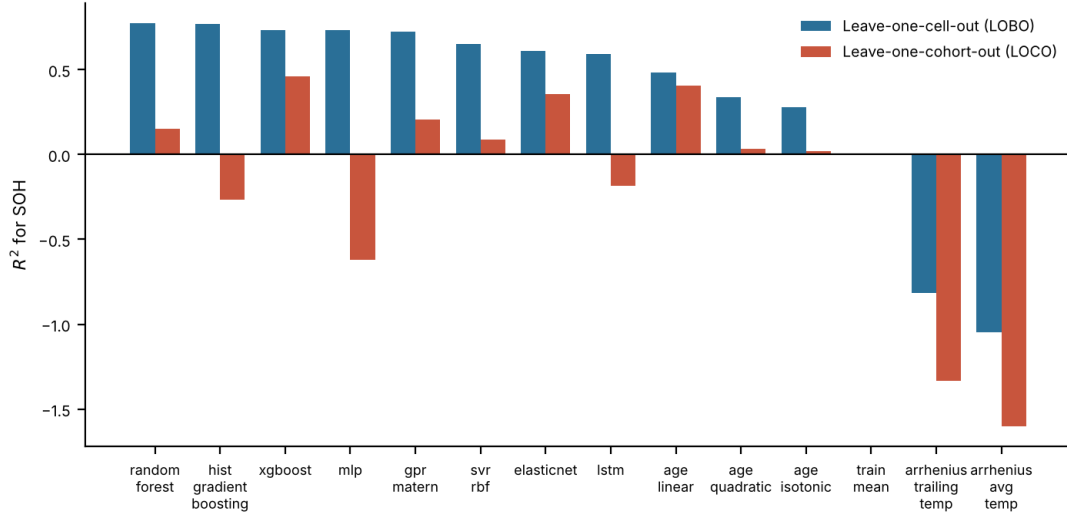


Figure 5.1: LOBO and LOCO R^2 for SOH for every scored method on the 31-cell NASA benchmark (data: `benchmark_results.csv`). Skill measured across cells does not carry over to a new protocol.

Table 5.2: Feature Ablation and Shuffle Control (NASA SOH, LOCO)

Run	Method	LOCO R^2	95% CI
All features (published)	XGBoost	0.459	$[-2.80, 0.86]$
All features (published)	Age-linear	0.406	$[-0.43, 0.51]$
Cycle number only	XGBoost	-0.266	$[-1.15, 0.82]$
Behaviour features only	XGBoost	-0.293	$[-0.72, 0.80]$
Shuffled in time (control)	Age-linear	-0.039	$[-0.35, -0.004]$
Shuffled in time (control)	XGBoost	-0.436	$[-1.44, -0.008]$

Temperature was the only usage factor with a consistent, correctly signed within-protocol relationship to fade (seven of seven cells in the two cohorts where it was significant), and its magnitude did not transfer between protocols. The rule-based risk score did not track measured fade either ($\rho = -0.27$, $p = 0.12$, on the 33 cells of the earlier health-index ranking), so it is labelled a heuristic.

Interpretation. No fitted method retains reliable skill under a change of protocol, and no method can be distinguished from another. This is the result that motivates the rest of the project: the estimators that ship fit nothing across cells.

5.2 FIELD STATE-OF-HEALTH ESTIMATION

Table 5.3 compares three ohmic treatments at the primary window on 22 CALCE cells. The field correction matched the laboratory correction within its interval and measured five more cells than no correction. Sensitivity windows gave 1.2% (4.05–3.55 V, 16 cells) and 1.9% (3.85–3.55 V, 17 cells).

The denominators differ because a correction can only be scored on cells for which

Table 5.3: Field SOH at the 3.90–3.60 V Window; Median Per-Cell Absolute Error Against Cyclers Capacity, 95% Bootstrap Interval Over Cells

Ohmic correction	Cells	Median	95% CI	Worst
None	12/22	4.3%	2.5–5.4%	6.4%
Cycler resistance (laboratory only)	12/20	1.4%	1.2–2.5%	5.6%
Step resistance (field)	17/21	1.7%	1.4–4.3%	10.3%

it exists: one cell never rests before discharging, so no step resistance exists for it, which gives 17 of 21 cells for the field arm and 17 of 22 once that cell is counted as refused. The three worst cells (6.4–10.3%) are all Type 3, whose discharge switches rate within a cycle. Every refusal carries a stated cause: four storage and partial-protocol cells delivered 80–155 equivalent full cycles before any constant-current discharge crossed the window, so the reference gate refused them, and one cell never rests before discharging. Fig. 5.2 tracks the estimate against every laboratory capacity test, and Fig. 5.3 shows every measured cell.

State of health from voltage, current and time only, against lab capacity tests

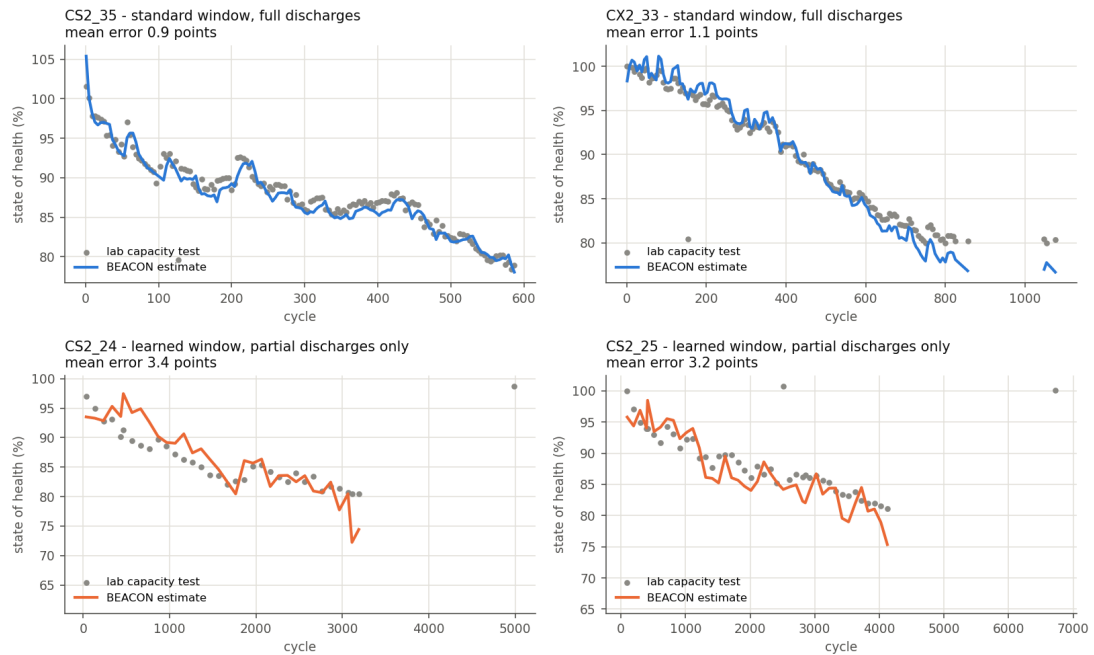


Figure 5.2: SOH from voltage, current and time against every laboratory capacity test. Top: two constant-current cells, fixed window (mean error 0.9 and 1.1 points). Bottom: two cells of real partial cycling, learned window, estimate from partial discharges only (3.4 and 3.2 points).

Interpretation. Identifying resistance from the load step took the median error from 4.3% to 1.7% and the coverage from 12 to 17 cells, without any quantity the cycler computed. Two earlier figures were corrected downward by this work: an earlier SOH figure and an 88% RUL figure had been scored against a reference capacity taken from each cell’s whole life, which uses the future and which no BMS can hold.

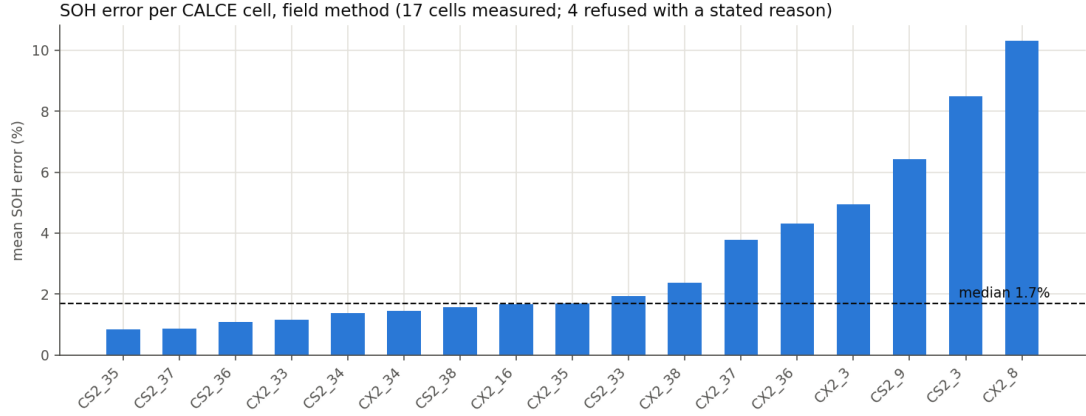


Figure 5.3: Mean absolute SOH error for every measured CALCE cell, field method; the dashed line is the median (1.7%).

5.3 PARTIAL-DISCHARGE SOH ESTIMATION

On the four cells of real partial cycling (Table 5.4), estimates use only the partial discharges preceding each laboratory check. Before the knee gate existed, the two near-empty cells were reported at 6.8% and 12.8% error while appearing confident. Learned windows on the 18 full-discharge cells have steepness 0.44–0.88, so any threshold between about 0.9 and 8 gives the same verdict on every cell and the chosen 2.0 is not tuned. As a control, the learned mode on those 18 cells measured 16 at a median of 1.95%, beside 1.7% for the fixed window, so it is a fallback and not a replacement.

Table 5.4: SOH from Real Partial Cycling (CALCE CS2 Types 5 and 6)

Cell	Band	Learned window	Steepness	Result
CS2_24	top of charge	4.00–3.82 V	0.83	3.4% error
CS2_25	top of charge	4.03–3.83 V	0.82	3.2% error
CS2_5	near empty	–	10.4	refused (knee)
CS2_6	near empty	–	8.2	refused (knee)

Interpretation. The refusal is physics and not a tuned cut. On the plateau, window charge tracks capacity, while on the knee voltage is set by resistance and diffusion, so it measures how hard the cell is working instead. The scope is two cells per band, which shows the mechanism and is too few to quote a population error.

5.4 INTERNAL RESISTANCE ESTIMATION

Against the cyclers’ internal-resistance column over each cell’s life (20 cells carrying both values), the step resistance achieved a median Spearman correlation of 0.86, growth in the same direction on 19 of 20 cells, and a median growth of 1.26 times against the cyclers’ 1.29 times. This includes the partial-cycling cells (0.90 and 0.98), and the exception is a Type 3 cell. The levels differ, since the step value includes early

polarisation, but the trends agree, which is what a power-fade indicator requires.

5.5 REMAINING USEFUL LIFE ESTIMATION

At threshold 0.90, referenced to each cell’s first cycles (17 cells), Table 5.5 reports accuracy by the true distance to end of life and by the prediction a user sees. Near end of life the estimate is a safe lower bound: it warns early and almost never late. Errors are predominantly early because these cells fade quickly at first and then more slowly, so a full-history line is steeper than the remaining fade.

Table 5.5: RUL to 90% Capacity. Left: By True Cycles to End of Life. Right: By Predicted Cycles, With the 90% Range of (True – Predicted)

True	Within ± 20	Median err.	Predicted	Lasted $\geq$ pred.	90% range
0–25	73%	9.6	0–25	95%	+1 to +82
25–50	44%	22.8	25–50	88%	–14 to +129
50–100	15%	42.7	50–100	85%	–40 to +141
200–400	0%	144	–	–	–

Scored against a reference capacity taken as the 95th percentile of the cell’s whole record, the same estimator was within ± 20 cycles 88% of the time near end of life. That reference uses the future, which no BMS can hold. Against the cell’s own early capacity the figure is 73%, and that is the figure reported. Fig. 5.4 summarises the result.

Remaining life to 90%: 73% within ± 20 cycles in the last 25 cycles (median miss 10 cycles)

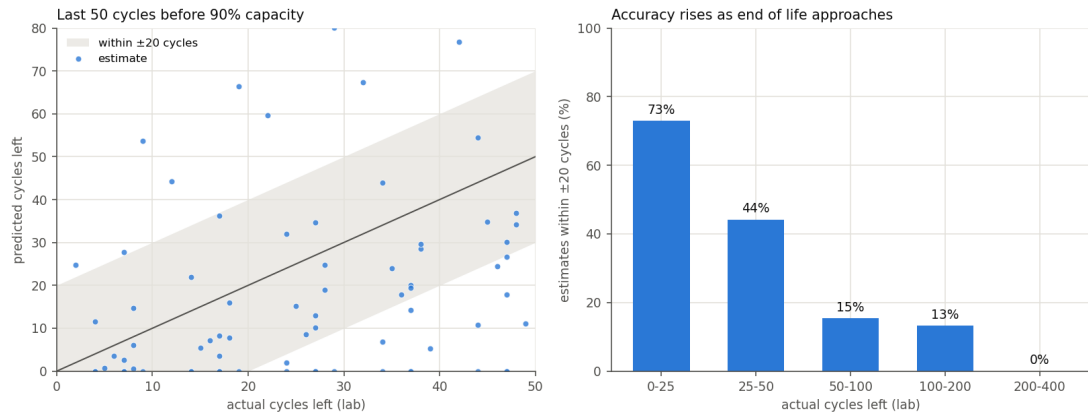


Figure 5.4: RUL to 90% capacity. Left: predicted against actual cycles left in the last 50 cycles. Right: share of estimates within ± 20 cycles by actual distance to end of life.

Interpretation. The “within ± 20 cycles” claim is true near end of life and false far from it (0% at 200 or more cycles out), so it is reported with its condition and not bare. The conservative direction is a property of the 90% threshold on these cells and is not a general guarantee. At an 85% threshold the median bias near end of life turns positive (+6.7 cycles within 25 cycles of end of life), so the estimate runs late there and only 62.8% of estimates fall within ± 20 cycles. The conventional 80% end-of-life

threshold is not validated on CALCE, because its full-discharge trajectories end near 0.81 and only one cell of 22 crosses 0.80.

5.6 RUL ESTIMATION UNDER SPARSE COMPLETE DISCHARGES

A driver who seldom fully discharges produces few complete discharges (on the partial-cycling cell CS2_24 they are rare). Four methods were evaluated on the same sparse evidence (Table 5.6). Sparsity is real on the partial-cycling cells and emulated as one complete discharge in 50 on the constant-current cells.

Table 5.6: RUL Estimation from Sparse Complete Discharges (Methods Compared)

Method	Outcome
Sparse capacity, standard minimum	Refuses every point (never reaches 30)
Field SOH trajectory	Answers 62%; median miss 79 cycles when predicting under 25
Field SOH scaled by the cell's own complete discharges	Over-corrects; errors turn late near end of life
Sparse capacity, minimum lowered to 5 (post hoc)	0% within ± 20 cycles; misses late

The failure of the field-trajectory method has a clean cause. Near 0.90, field SOH reads a median 1.1 points below capacity (measured in a first pass of this study, not kept as a separate artifact). Because fade near the threshold is slow, that small bias moves the 0.90 crossing a median 56 cycles early. An SOH error that is acceptable as a health figure thus becomes large as a time to threshold. None of the four is reported, and the system refuses RUL for partial-only records and states that this was tested.

5.7 EVIDENCE SUFFICIENCY ANALYSIS

Three questions were fixed before the study and scored at every laboratory check on 19 cells. First, *count does not predict accuracy*: median error was 0.75% at 6–10 usable readings and 1.97% beyond 60, because error grows with age as the approximation in Eq. (3.1) drifts. A rule of high confidence after a fixed number of discharges therefore has no support. Second, *consistency does* (Table 5.7). The pooled Spearman correlation between u and absolute error was 0.48. Within cells it was weaker (median 0.14, positive in 14 of 19), so u chiefly separates unreliable cells and periods and does not rank readings within a good one.

Table 5.7: Error by the Consistency Criterion of Eq. (3.7)

Readings	Points	Cells	Median error	90th percentile
$u \leq 0.01$ (consistent)	8,993	16	1.3%	4.4%
$u > 0.01$ (inconsistent)	2,168	13	5.1%	21.9%

Third, *reference size*: in the study’s own scoring, larger references reduced error (one discharge 3.6%, five 2.0%, ten 1.3%), but on the field-study metric ten versus five changed the median only from 1.681% to 1.665%, so the estimator retains five.

5.8 EXTERNAL VALIDATION ON THE OXFORD DATASET

The estimator was applied to the eight Oxford cells without re-tuning. Arms, reference and scoring were fixed before the run. The reference is each cell’s first characterisation, and truth is 1C capacity relative to it. Because each characterisation block begins under load, no rest-to-load step exists, so the 1C window is uncorrected.

Table 5.8: External Validation on Eight Oxford Cells: Median and Worst Per-Cell Mean Absolute SOH Error

Window	Cells	Median	Worst
Fixed 3.90–3.60 V, 1C discharge	8/8	3.0%	4.0%
Fixed 3.90–3.60 V, C/18 pseudo-OCV	8/8	3.2%	3.4%
Learned window, 1C discharge	8/8	2.3%	3.6%

All eight cells were measured (Table 5.8). The near-equilibrium C/18 window was not more accurate than the 1C window, so on this blend cathode the residual error stems from the curve changing shape with age and not from ohmic sag, consistent with the stated approximation of Eq. (3.1).

Drive-Cycle Discharge. The drive-cycle discharge of Cell 1 has current between -5.0 and $+1.6$ A. Resistance from its own 1,575 current steps of at least 0.2 A gave a median of 21 m Ω (interquartile 19–24). The discharge is itself partial (it stops at about 65% depth), so a coverage window (4.08–3.78 V) was chosen from the corrected curve’s span alone, before any comparison was computed. In it the drive-cycle reading, corrected per sample and fitted monotonically, was 0.2625 Ah, against 0.2661 Ah from the same cell’s 1C constant-current discharge, a difference of 1.4%. Both lay about 13% below the C/18 reading in that window, so the rate effect cancels in a ratio of like with like but would bias one that mixed rates. This is a single discharge and demonstrates the mechanism, not a population error.

Remaining Life at the 80% Threshold. The Oxford trajectories cross the 80% convention, so the shipped RUL estimator was scored there unchanged. When it predicted fewer than 500 cycles to 80%, the cell lasted at least that long in 97% of cases (36 estimates, 5 cells). Precision is low: the median miss is 385 cycles against characterisations 100 cycles apart. The guarantee also weakens with distance, since predictions of 500 to 1,000 cycles were met or exceeded only 80% of the time (25 estimates, 5 cells).

5.9 CROSS-DATASET VALIDATION ON THE NASA DATASET

The NASA files (18650 cells, ambient 4–44 °C) were read from their original MATLAB form and scored the same way, unchanged. The estimator was poor on NASA overall, with a median per-cell error of 9.2% over 27 cells, and it depended on temperature: 1.7% in the 43 °C cohort and 12–14% at 4 and 22 °C. The step resistance captures only the ohmic part of the overpotential, and at low temperature the charge-transfer and diffusion parts, which grow with age, move the curve through the window.

The consistency uncertainty, with its tolerance fixed on CALCE, was applied unchanged. On NASA, estimates it labelled HIGH or MEDIUM had a median per-cell error of 4.5%, while those labelled LOW had 13.2% (within three SOH points: 49% versus 12%). The failure on unseen conditions was thus largely signalled by the estimator itself.

For contrast, a gradient-boosted model on three window features, the setting where fitting looks best, reached 0.7% under leave-one-cell-out within Oxford and 1.2% within CALCE, but 6.7% when moved from Oxford to NASA and 5.5% from CALCE to Oxford. Validation inside one dataset overstated transferable accuracy four- to tenfold.

5.10 SYSTEM AND HARDWARE RESULTS

5.10.1 Bench Rig Captures

Three captures are discussed here (Table 5.9); the repository holds further bring-up files, including one garbled capture from the baud-rate fault and one longer synthetic run. The first is a counter-example: a board with no sensors attached, whose synthetic profile still scores, and which is kept and labelled as such.

Table 5.9: Bench Rig Captures

Capture	Declared channels	Records	Measured
Stage A (no sensors)	all, from stubs	151	Synthetic, not a hardware result
Stage B voltage	t, v, i , SOC (INA219 up, LM35 refused)	83 over 82 s	3.9871 V (sd 1.9 mV); current at the ± 0.4 mA noise floor
Temperature demo	t, t_c (LM35 up, INA219 refused)	85 over 84 s	32.11 °C (sd 0.086 °C)

In both real captures the cadence held at 1.000 s with no gaps and a monotonic time base. Every measured voltage is an exact multiple of the INA219’s 4 mV bus LSB, which is what a genuine read from that part looks like and what a fabricated one would not. The sensor-complement declaration worked as designed: the firmware named the sensor that failed at boot, declared only the channels it could supply, and the host scored those and refused the rest by name. The first real board also exposed a baud-rate fault (the CH340 bridge accepts only 9600 and 19200 baud), which the refusal wording named

as a cause and which led to the fix.

Fig. 5.5 plots the Stage B capture and Fig. 5.6 shows how the dashboard presents it.

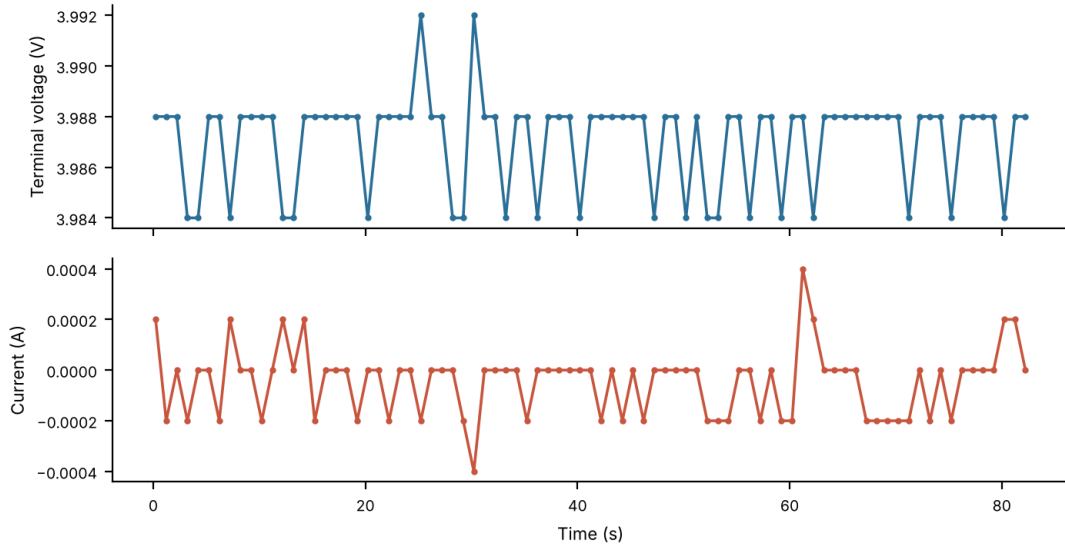


Figure 5.5: Stage B capture from the bench rig (83 frames over 82 s, one cell at rest). Voltage moves between 3.984 and 3.992 V in steps of the INA219’s 4 mV LSB, and current stays at the ± 0.4 mA noise floor.

5.10.2 Fault Injection

Faults injected into a real capture behaved as designed: a corrupted byte failed its checksum, a NaN field was rejected, a reversed timestamp refused the capture, and a missing sensor was refused by name. The injection also exposed one defect, since corrected: a 999 V reading passed the field range, which is wide so that packs are admissible. Records outside 0–5 V are now rejected when the source declares a single cell.

5.10.3 Scope of the Hardware Results

The captures demonstrate that the transport, the protocol, the gate and the refusal path work against real silicon. They are one cell, at rest, for under 90 s, with no load step and no charge or discharge cycle. They are *not* a validation of the health, RUL or risk models, which were fitted on cycled research cells and cannot be confirmed or refuted by a cell that was never cycled. Sensor accuracy against a reference meter was not measured, and no discharge has yet been recorded on the rig.

5.11 CASE STUDY: HEALTH REPORT CARD

Fig. 5.7 abridges the report card for CALCE cell CS2_35 as it would have read at cycle 90. The pipeline saw voltage, current and time only.

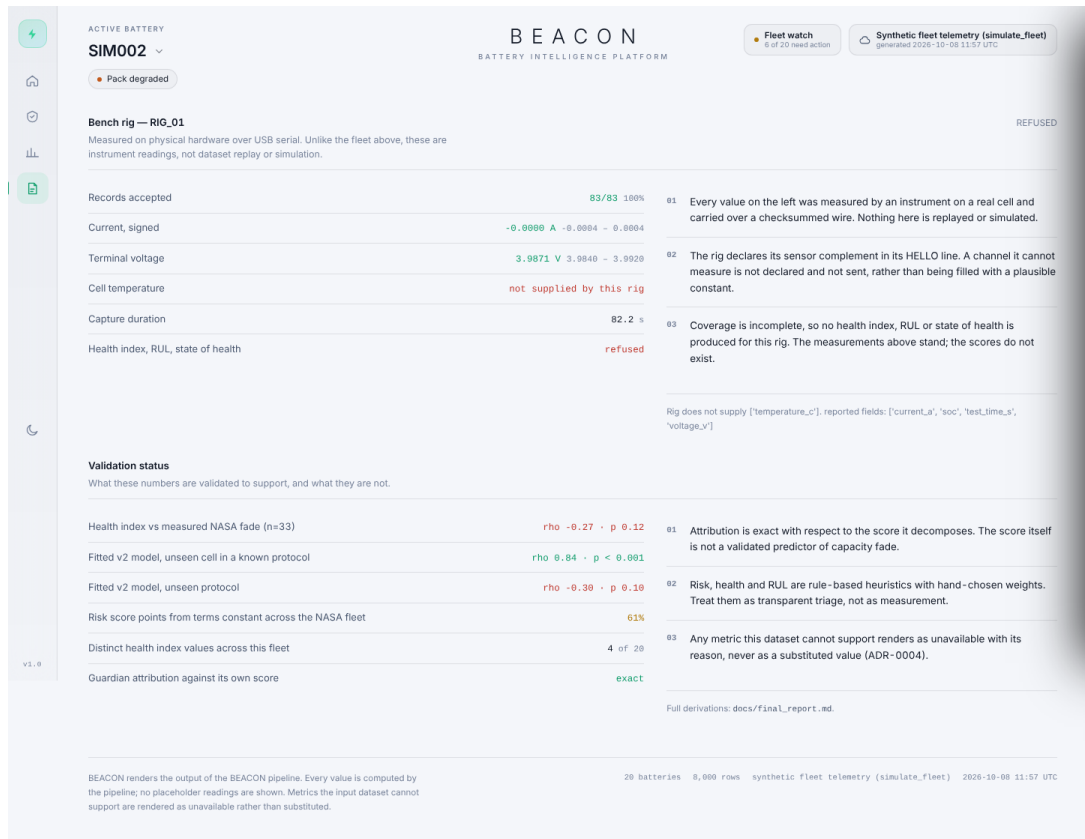


Figure 5.6: Evidence view of the BEACON dashboard (light theme) for the bench rig: 83 of 83 records accepted, voltage and current as measured, temperature reported as not supplied, and the health index, remaining life and state of health refused because coverage is incomplete.

1. HOW HEALTHY IT IS
91.0% state of health [MEASURED] State: WARNING
2. HOW LONG IT HAS LEFT
About 5 cycles until it falls to 90% of its early capacity. [ESTIMATE]
3. WHAT YOUR USAGE IS DOING TO IT
No temperature was measured, so heat exposure cannot be assessed.
4. WHAT TO DO
Capacity fade is measurable. Keep monitoring.
5. WHAT THIS REPORT CANNOT TELL YOU
- Scoring skipped: required channels not supplied.

CHECK AGAINST THE LAB (the card was not shown this)
Lab capacity at cycle 90: 91.4%; card said 91.0%
Cell actually crossed 90% at cycle 146: 56 cycles
after cycle 90; card said 5

Figure 5.7: Abridged health report card for CS2_35 at cycle 90, with the laboratory check.

The health figure is within half a point. The remaining-life figure is 51 cycles early, which is the conservative bias the RUL study measured at that distance (median –32 cycles at 50–100 cycles out), and it is the reason the card prints its own accuracy beside the number and not the number alone. Only temperature is offered as a usage finding, because it is the one behaviour-to-ageing link that survived testing, in direction only.

5.12 DISCUSSION

Effect of Conditioning Versus Model Choice. Across fourteen scored methods, LOCO differences were smaller than the uncertainty on any one of them. Three conditioning decisions, by contrast, changed error by multiples. Refusing truncated discharges took the worst cell from 52% to about 10%. Identifying resistance from the load step took the median from 4.3% to 1.7% and coverage from 12 to 17 cells. Refusing knee windows converted 6.8–12.8% errors into refusals. For a BMS software layer, the leverage lies in what is measured and how it is conditioned, not in the regressor that follows. The first of these was a misdiagnosis corrected during the project: the 52% worst case had been attributed to loss of active material, when it was actually truncated discharges being read as full ones.

Dependence on the Reference Definition. Two results moved when the reference was made one a BMS can hold: RUL accuracy fell from 88% to 73%, and a partial-discharge reference formed from a late discharge produced a large error until the timing gate existed. Validation should use the reference the deployed system will have.

Sensitivity of Threshold Crossings to Bias. A 1.1-point SOH bias, unremarkable as a health figure, shifted predicted end of life by about 56 cycles. Any RUL built on an estimated rather than measured SOH inherits this, and should be evaluated as a time to threshold and not through SOH error.

Quantifying Evidence Sufficiency. Treating “enough data” as a count is intuitive and here wrong, because the estimator’s error is dominated by age-related drift and not sampling. The consistency of readings is observable without ground truth and separates 1.3% from 5.1% median error, which gives a BMS layer a principled way to report “not yet” for a battery it has never seen.

Limitations of the Findings.

- *Temperature.* On NASA cells at 4 and 22 °C the estimator erred by 12–14%. The low-confidence label marked most of these readings, but a temperature-aware correction is future work.

- *Two cathode families, single cells, one real drive cycle.* Packs add cell-to-cell imbalance, thermal gradients and balancing currents, and the flat plateau of lithium iron phosphate would challenge the window method directly.
- *Two cells per partial-cycling band.*
- *End of life:* validated at 90% on CALCE, and at 80% on Oxford only as a conservative bound with low precision.
- *Apparent resistance* depends on the logger's sample interval and is comparable within a record, not across loggers.
- *No hardware discharge* has yet been recorded on the bench rig, and the heuristic risk layer is not validated.

5.13 IMPLICATIONS FOR BMS SOFTWARE DESIGN

Four implications follow from the results and are stated here as design guidance for a software layer above a BMS.

Validation Unit. A model is deployed on a battery that has not been seen and often on a protocol that has not been seen. The validation split should hold out that unit, and the interval around the result should be reported, because on the benchmark of this project the interval on a single method was wider than the difference between methods.

Measurement Versus Fitting. Voltage, current and time are enough to measure capacity on a discharge that spans a voltage window, and the measurement needs no training cells. Fitting is then needed only for the quantities that cannot be measured directly.

Refusal as an Output. A refusal that carries its cause is information, since it tells the operator what data to collect. The consistency criterion separated a 4.5% median error from a 13.2% median error without any knowledge of the true capacity, which shows that a system can recognise its own poor conditions.

Single Scoring Path. When a vehicle, a rig and a dataset all pass through the same function, a result obtained on a dataset says something about the vehicle, and a defect found on the rig is fixed for all three.

5.14 VALIDATION SUMMARY

Table 5.10 lists the outputs by status. Claims that failed or were never tested are listed, because that is what makes the others believable.

Table 5.10: Status of the Main Outputs

Output	Result	Status
Capacity health (full discharges)	1.7% median, 17 of 22 cells	Validated
Capacity health (partial discharges)	3.2–3.4%, 2 cells	Validated, 2 cells
Resistance health	Same direction on 19 of 20 cells	Validated
RUL near end of life	73% within ± 20 cycles	Validated, 90% threshold
RUL far from end of life	0% at 200–400 cycles	Not valid (shown with band)
RUL from sparse complete discharges	Four methods, none accurate	Refused
Second dataset (Oxford)	3.0% median, 8 of 8 cells	Validated
Third dataset (NASA)	9.2% overall, 1.7% at 43 °C	Fails outside warm conditions
Fitted models across protocols	Best LOCO R^2 0.459, CI [−2.80, 0.86]	Not generalising
Heuristic risk score	$\rho = -0.27$, $p = 0.12$	Labelled heuristic
Telemetry on hardware	83/83 frames, 1.000 s cadence	Validated (acquisition only)
Health from a real rig discharge	No discharge recorded	Not tested
Packs, LFP, large-scale drive cycles	No data	Not tested

5.15 ASSESSMENT AGAINST THE OBJECTIVES

Table 5.11 relates each objective of Section 1.5 to the evidence for it in this chapter and in Chapter 4.

Table 5.11: Objectives and Evidence

No.	Objective	Evidence
1	End-to-end ingestion of CAN, serial and dataset sources through one pipeline	Single scoring function; three transports; test modules and continuous integration
2	SOH from voltage, current and time with ohmic correction	1.7% median error on 17 of 22 CALCE cells
3	Resistance as a second quantity; RUL from each cell's own fade	Resistance trend in the same direction on 19 of 20 cells; 73% of RUL within ± 20 cycles near end of life
4	Measured evidence sufficiency; refusal of insufficient evidence	Consistency criterion separates 4.5% from 13.2% error; every refusal carries a reason
5	External validation without re-tuning; retention of negative results	Oxford median error 3.0%; sparse-discharge RUL and fitted-model ranking retained as negative results
6	Physical rig, API and dashboard	83 of 83 frames received; routes and dashboard served from containers

5.16 THREATS TO VALIDITY

The results are bounded by the way they were obtained, and the bounds are stated here so that they can be weighed against the figures.

- **Chemistry and format.** Development and primary validation use LCO prismatic cells at constant current. The external dataset is an NMC/LCO blend pouch cell at one temperature. Results for lithium iron phosphate, for packs and for dynamic loads are not claimed.
- **Reference definition.** Two results moved when the reference was changed to the capacity the cell's own first discharges measure, which is what a BMS can know. A figure is therefore tied to the reference named with it.
- **Small numbers of cells.** Several results rest on a few cells, for example the partial-discharge result on two cells. They are reported as validated on those cells, and no claim is made beyond them.
- **Hardware.** The rig validates the data-acquisition path. No discharge has been recorded on it, so no health figure from the rig is claimed.
- **Heuristic layer.** The behaviour risk score was not significantly related to measured fade ($\rho = -0.27$, $p = 0.12$ on the 33 cells of the earlier health-index ranking of the project, a slightly larger set than the 31-cell benchmark) and is labelled a heuristic wherever it appears.

5.17 ROBUSTNESS AND SECURITY CHECKS

Beyond estimation accuracy, the system was checked for the behaviour of its interfaces under bad input. The serial parser ignores lines without the sentinel as noise, counts and discards malformed lines, rejects out-of-range, non-finite and checksum-failed values instead of clamping them, and refuses a capture whose accepted fraction falls below a configured threshold. These behaviours are exercised by tests with truncated lines, corrupted checksums, non-finite values, out-of-range fields and unknown record types. The path-containment fix was verified by thirteen tests, and a search of the source files for keys, secrets and language-model clients found none, apart from a benign file name in a test. Continuous integration runs with no key, no hardware and no external service beyond the package registries, so the checks run the same way on every commit.

CHAPTER 6

CONCLUSION AND FUTURE WORK

This project asked whether a software layer above a battery management system can turn voltage, current and time into a trustworthy battery health report, and when it should decline to answer. It built BEACON, a three-tier system that ingests telemetry from a CAN bus, a USB serial rig or research datasets, converges all three on one schema, and scores them through one shared pipeline that is served to a React dashboard.

The research answer is that such a layer can measure cell health without any model trained on other cells. On 22 CALCE cells it delivered capacity health at 1.7% median error (95% CI 1.4–4.3%) on 17 cells, with every one of the five refusals explained, and a resistance trend that tracked the laboratory instrument in the same direction on 19 of 20 cells. Partial-discharge capacity health reached 3.2–3.4% on two cells of real top-of-charge cycling, with a physical rule that refuses windows on the discharge knee. Unchanged, the estimator measured all eight cells of an independent NMC/LCO dataset at 40 °C with a 3.0% median error, and a real drive-cycle discharge read within 1.4% of a constant-current one. Remaining useful life extrapolated from each cell’s own fade was within ± 20 cycles 73% of the time in the last 25 cycles before 90% capacity, and at that threshold behaved as a safe lower bound.

Equally important are the results that failed. Eleven of twelve methods, all but a mean-only baseline, lost skill when the held-out unit changed from a cell to a protocol cohort, and no ranking of methods was supportable, which is why the shipped estimators fit nothing across cells. RUL from sparse complete discharges was tested by four methods and refused. The estimator was poor on cold NASA cells (12–14%), but its own consistency criterion, with a tolerance fixed on CALCE, marked those failures: 4.5% median error when it said high or medium confidence against 13.2% when it said low.

On the engineering side, the system refuses to output a number it cannot justify. The gates return a refusal as a value instead of an exception, the wire protocol is self-synchronising with record-level and capture-level checks, and 44 test modules and an eight-job CI pipeline pin the documentation to the artifacts that produced it. A physical rig built on a NodeMCU ESP8266 with an INA219 and an LM35 streamed 83 of 83 checksummed frames at a measured 1.000 s cadence. That validates data acquisition. It does not validate health estimation, because no discharge has yet been recorded on the rig.

Each output therefore carries a measured error band and is withheld when its evidence is missing or inconsistent, and the criterion that makes that decision was itself

validated against laboratory truth.

6.1 SUMMARY OF THE WORK

The project proceeded in four stages. The first built the data pipeline and a behaviour layer of rules, and found by benchmarking that fitted models did not hold up when the test protocol changed. The second replaced fitted models with estimators that measure each cell against its own behaviour, validated them on CALCE, and added a criterion for when to refuse. The third tested the estimators on a second and a third dataset without re-tuning, and built the CAN, serial and dataset paths into one scoring function. The fourth built the bench rig, the service tier and the dashboard, and fixed the documentation to the artifacts that produced it, so that every number in the report can be traced to a script and a file.

6.2 LIMITATIONS

The scope is single cells: LCO for development, an NMC/LCO blend pouch for external validation, mostly at constant current, with one real drive-cycle discharge. Packs, lithium iron phosphate and temperature-dependent capacity correction are not covered. The conventional 80% end-of-life threshold is not validated on CALCE, and the Oxford results at 80% are a conservative bound with low precision. The heuristic risk layer is not validated against measured degradation. The bench rig has been run on one cell, at rest, for under 90 s, and its two sensors have not yet operated in the same capture.

6.3 FUTURE WORK

In order of dependency:

1. **Hardware validation.** Capture a load step and a full constant-current discharge on the rig with all three channels real, compare the resulting capacity with an independent charger/discharger, and measure sensor accuracy against a reference meter. Then repeat on more than one cell.
2. **Protocol hardening.** Replace the XOR checksum with a CRC-8 at the same cost, which detects burst errors and byte transpositions the XOR form cannot.
3. **Removing the remaining silent default.** The serial path no longer assumes a rated capacity, but the CAN and batch paths still do. A per-vehicle registry entry is needed before the CAN path is pointed at a real vehicle.
4. **Temperature-aware correction.** Extend the ohmic correction to the charge-transfer and diffusion terms that dominate at low temperature, so that the cold-cell failure is reduced and not only flagged.

5. **Broader validation.** Test packs, lithium iron phosphate and large-scale dynamic drive cycles, and obtain data that crosses the 80% end-of-life convention for a validated result there.
6. **Persistence and operations.** Add a database behind the fleet store, structured logging and metrics, a load-test harness, and asynchronous serial ingest with backpressure.

6.4 LESSONS LEARNED

Several lessons from the project apply beyond it. A validation split is part of the claim, and a result that is stated without it is incomplete. A default value is a hidden measurement, and removing defaults, as with the rated capacity that is now declared, closed a class of silent errors. Reading code and documents for defects before connecting hardware is cheaper than finding the same defects with a cell on the bench. A negative result retained with its evidence, such as the failure of sparse-discharge RUL, is more useful to a later reader than a positive result with no account of what was tried first.

6.5 RECOMMENDATIONS FOR DEPLOYMENT

The system as built is a bench and analysis tool, and moving it beyond that needs the steps below, which follow from the limitations and from the security review.

1. Place an authenticating reverse proxy with transport security and rate limiting in front of the service, and restrict the allowed data roots to the deployment's own directories through the environment variable provided for that.
2. Add a persistence layer behind the fleet store, so that the twin history survives a restart and more than one process can share it.
3. Register a rated capacity and a verified DBC file for each vehicle, so that neither the reference capacity nor the signal mapping is assumed.
4. Record a load step and a full discharge for a few cells of the target chemistry before relying on any health figure, so that the measured error bands can be confirmed for that chemistry.
5. Keep the displayed provenance and the refusals visible to operators, because they are what tells an operator how far to trust a figure.

REFERENCES

- [1] W. Waag, C. Fleischer, and D. U. Sauer, “Critical review of the methods for monitoring of lithium-ion batteries in electric and hybrid vehicles,” *J. Power Sources*, vol. 258, pp. 321–339, 2014, doi: 10.1016/j.jpowsour.2014.02.064.
- [2] M. Berecibar, I. Gandiaga, I. Villarreal, N. Omar, J. Van Mierlo, and P. Van den Bossche, “Critical review of state of health estimation methods of Li-ion batteries for real applications,” *Renew. Sustain. Energy Rev.*, vol. 56, pp. 572–587, 2016, doi: 10.1016/j.rser.2015.11.042.
- [3] J. Vetter *et al.*, “Ageing mechanisms in lithium-ion batteries,” *J. Power Sources*, vol. 147, pp. 269–281, 2005, doi: 10.1016/j.jpowsour.2005.01.006.
- [4] A. Barré, B. Deguilhem, S. Grolleau, M. Gérard, F. Suard, and D. Riu, “A review on lithium-ion battery ageing mechanisms and estimations for automotive applications,” *J. Power Sources*, vol. 241, pp. 680–689, 2013, doi: 10.1016/j.jpowsour.2013.05.040.
- [5] G. L. Plett, “Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs,” *J. Power Sources*, vol. 134, pp. 277–292, 2004, doi: 10.1016/j.jpowsour.2004.02.033.
- [6] K. A. Severson *et al.*, “Data-driven prediction of battery cycle life before capacity degradation,” *Nature Energy*, vol. 4, no. 5, pp. 383–391, 2019, doi: 10.1038/s41560-019-0356-8.
- [7] K. Maher and N. Yerken, “Comprehensive machine learning for lithium-ion battery state-of-health estimation using group-wise cross-validation,” in *Proc. 14th Int. Conf. Renew. Energy Res. Appl. (ICRERA)*, IEEE, 2025, pp. 1465–1468, doi: 10.1109/ICRERA66237.2025.11283788.
- [8] H. H. Le and K.-A. Nguyen, “Charging phase health indicators for battery state-of-health estimation: A systematic comparison of CC, CV, and combined approaches under cross-battery validation,” arXiv:2607.23482, 2026.
- [9] I. Bloom *et al.*, “Differential voltage analyses of high-power, lithium-ion cells,” *J. Power Sources*, vol. 139, pp. 295–303, 2005, doi: 10.1016/j.jpowsour.2004.07.021.

- [10] M. Dubarry, C. Truchot, and B. Y. Liaw, “Synthesize battery degradation modes via a diagnostic and prognostic model,” *J. Power Sources*, vol. 219, pp. 204–216, 2012, doi: 10.1016/j.jpowsour.2012.07.016.
- [11] C. Weng, Y. Cui, J. Sun, and H. Peng, “On-board state of health monitoring of lithium-ion batteries using incremental capacity analysis with support vector regression,” *J. Power Sources*, vol. 235, pp. 36–44, 2013, doi: 10.1016/j.jpowsour.2013.02.012.
- [12] C. R. Birkl, M. R. Roberts, E. McTurk, P. G. Bruce, and D. A. Howey, “Degradation diagnostics for lithium ion cells,” *J. Power Sources*, vol. 341, pp. 373–386, 2017, doi: 10.1016/j.jpowsour.2016.12.011.
- [13] J. Tian, R. Xiong, W. Shen, J. Lu, and X.-G. Yang, “Deep neural network battery charging curve prediction using 30 points collected in 10 min,” *Joule*, vol. 5, pp. 1521–1534, 2021, doi: 10.1016/j.joule.2021.05.012.
- [14] X. Hu, L. Xu, X. Lin, and M. Pecht, “Battery lifetime prognostics,” *Joule*, vol. 4, pp. 310–346, 2020, doi: 10.1016/j.joule.2019.11.018.
- [15] B. Saha and K. Goebel, “Battery data set,” NASA Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA, USA, 2007.
- [16] Y. Liu, S. Saxena, M. Pecht, *et al.*, “CALCE CS2 and CX2 prismatic cell cycling series,” Center for Advanced Life Cycle Engineering, University of Maryland, College Park, MD, USA.
- [17] D. Howey and C. Birkl, “Oxford battery degradation dataset 1,” University of Oxford, 2017, doi: 10.5287/bodleian:KO2kdmYGg.
- [18] C. R. Birkl, “Diagnosis and prognosis of degradation in lithium-ion batteries,” Ph.D. dissertation, Dept. Eng. Sci., Univ. Oxford, Oxford, U.K., 2017.
- [19] ISO 11898-1:2015, Road vehicles — Controller area network (CAN) — Part 1: Data link layer and physical signalling,” International Organization for Standardization, Geneva, Switzerland, 2015.
- [20] M. Grieves, Digital twin: Manufacturing excellence through virtual factory replication,” White Paper, 2014.
- [21] V. Pop, H. J. Bergveld, D. Danilov, P. P. L. Regtien, and P. H. L. Notten, *Battery Management Systems: Accurate State-of-Charge Indication for Battery-Powered Applications*. Dordrecht, The Netherlands: Springer, 2008.

- [22] G. L. Plett, *Battery Management Systems, Volume I: Battery Modeling*. Norwood, MA, USA: Artech House, 2015.

APPENDIX A

DATA AVAILABILITY STATEMENT

The three public datasets used in this work are available from their providers and are not redistributed with the project. The NASA Prognostics Center of Excellence battery data set [15] is published in the NASA Prognostics Data Repository. The CALCE CS2 and CX2 prismatic cell cycling series [16] is published by the Center for Advanced Life Cycle Engineering at the University of Maryland. The Oxford Battery Degradation Dataset 1 [17] is published by the University of Oxford under the digital object identifier 10.5287/bodleian:KO2kdmYGg. Each dataset carries the terms of its own provider, which apply to any reuse. After download, the archives are placed in `data/raw/nasa/`, `data/raw/calce/` and `data/raw/oxford/`, which the repository excludes from version control. When a raw archive is absent, the validation script skips the stages that need it and states the reason, and does not fail.

The results derived from these datasets are available in the project repository. Every metric, table and per-cell result quoted in this report is stored as a text file under `reports/metrics/`, and the figures built from them are stored under `reports/figures/`. A reader can therefore check each reported number without downloading any raw data, and can regenerate the files from the raw data with the scripts listed in Appendix B.

The bench-rig captures recorded by the authors, including the two real captures discussed in Chapter 5, are committed as text files under `data/interim/`, together with the hardware telemetry log kept in the repository. They were recorded from a single commercial 18650 cell at rest and contain voltage, temperature and timing values only. They contain no personal data and no data from any private vehicle or fleet. The fleet shown by the dashboard in Chapter 4 is produced by the project's own simulator and is labelled as simulated wherever it is displayed. The CAN decoding example is based on the publicly documented message of the Open Vehicles Monitoring System for the Renault Twizy.

The source code of the system, the firmware, the tests, the architecture decision records and the documentation are available in the project repository at <https://github.com/Navvu-github/Behavior-Aware-BMS> under the MIT licence. The system runs on a fresh clone without hardware, a database, an access key or a dataset download, as described in Appendix C.

APPENDIX B

PROVENANCE OF REPORTED RESULTS

Almost every number in this report is regenerated from a committed file in `reports/metrics`. A small number of figures from earlier stages of the study are recorded only in the project documents (the validation matrix, the panel-defence notes and the field-SOH manuscript draft under `docs/`) and not in a regenerated metrics file: the 1.665% median for a ten-discharge reference, the 52% to about 10% change for the worst cell before and after the truncated-discharge fix, the 6.8% and 12.8% errors before the knee gate, and the 1.1-point bias and 56-cycle early crossing in the sparse-RUL study. They are quoted as recorded there.

Every number in this report is regenerated by a named script from committed artifacts. Table B.1 maps each results section to the artifact under `reports/metrics/` and the script under `scripts/` that produced it.

Table B.1: Artifacts and Scripts Behind Each Results Section

Section	Artifact	Script
5.1	<code>benchmark_results.csv</code> , <code>ablation/</code>	<code>run_benchmark_study.py</code> , <code>run_ablation_study.py</code>
5.2	<code>calce_field_soh/</code>	<code>run_field_soh_study.py</code>
5.3	<code>calce_partial_soh/</code>	<code>run_partial_soh_study.py</code>
5.4	<code>calce_resistance/</code>	<code>run_resistance_study.py</code>
5.5	<code>calce_rul_horizon_early_ref/</code> , <code>error_bands.csv</code>	<code>run_rul_horizon_study.py</code> , <code>build_error_bands.py</code>
5.6	<code>calce_field_rul/</code>	<code>run_field_rul_study.py</code>
5.7	<code>calce_sufficiency/</code>	<code>run_sufficiency_study.py</code>
5.8	<code>oxford/</code>	<code>run_oxford_study.py</code> , <code>run_oxford_drive_cycle.py</code>
5.9	<code>cross_dataset/</code>	<code>run_cross_dataset_study.py</code>
5.10	<code>data/interim/rig_*.txt</code>	<code>run_serial_demo.py</code> , <code>panel_demo.py</code>
5.11	Console output of the report card	<code>health_report.py</code>
Figs. 5.2, 5.3, 5.4	<code>reports/figures/</code> <code>results_*.png</code>	<code>export_results.py</code>

APPENDIX C

REPRODUCTION PROCEDURE

The repository is <https://github.com/Navvu-gityhub/Behavior-Aware-BMS>. The commands below run on a fresh clone with no hardware, no database, no API keys and no dataset download.

```
git clone \
    https://github.com/Navvu-gityhub/Behavior-Aware-BMS.git
cd Behavior-Aware-BMS
pip install -r requirements-dev.txt

python main.py # end-to-end run
python scripts/run_serial_demo.py # emulated rig
python scripts/health_report.py --serial \
    data/interim/rig_stage_b_voltage_verified.txt
python -m pytest tests/ -q # the test suite
docker compose up # API, gateway, client
```

With the CALCE data downloaded, the report card of a real cell as it read at a given cycle is printed by

```
python scripts/health_report.py --calce \
    data/raw/calce/CS2/Type2/CS2_35.zip --upto-cycle 90
```

The raw CALCE, NASA and Oxford data are not redistributed; their sources are given in Appendix A.